# Minecraft Alpha 1.2.6

## Guía Completa de Referencia y Arquitectura del Motor (MCP)

Informe técnico exhaustivo, catálogo analítico de todas las clases y manual práctico de modificación para el código fuente deofuscado con Minecraft Coder Pack.

Minecraft Alpha 1.2.6 | Minecraft Coder Pack | 457 Clases Java | Referencia Modding

---

**Entorno:** Java SE / LWJGL 2 / MCP Decompiled

**Alcance:** Cobertura del 100% (457 archivos Java documentados)

**Fecha de Generación:** Octubre 2026

**Estructura:** net.minecraft.client (2 clases) • net.minecraft.src (455 clases)

# Tabla de Contenidos


1. Introducción y Contexto Histórico — Sección 1
2. Arquitectura Global del Motor y Ciclos Fundamentales — Sección 2
3. Guía Rápida de Modificación (Cheat Sheet & Modder's CookBook) — Sección 3
4. Catálogo Exhaustivo de los 13 Subsistemas (457 Clases) — Sección 4
    - 01. Núcleo y Ciclo del Juego (Core Game Loop & Engine) — 34 clases
    - 02. Bloques y Materiales (Blocks, Materials & Mechanics) — 66 clases
    - 03. Objetos, Crafteo e Inventarios (Items, Crafting & Inventory) — 47 clases
    - 04. Entidades, Mobs y Físicas (Entities, Living Mobs, Vehicles & AI) — 60 clases
    - 05. Generación de Mundo y Biomas (World Generation & Biomes) — 19 clases
    - 06. Mundo, Chunks, Iluminación y Guardado (World, Chunks, Lighting & Storage) — 30 clases
    - 07. TileEntities y Contenedores (TileEntities & Containers) — 13 clases
    - 08. Motor de Renderizado y Gráficos (Rendering Pipeline, Models & FX) — 83 clases
    - 09. Interfaz de Usuario y Menús (GUI, HUD & Screens) — 30 clases
    - 10. Redes y Protocolo Multijugador (Networking & Packets) — 46 clases
    - 11. Sistema NBT y Serialización (Named Binary Tag) — 13 clases
    - 12. Motor de Audio y Sonido (Sound Engine & Music) — 5 clases
    - 13. Matemáticas, Física y Estructuras de Datos (Math & Fast Collections) — 11 clases
5. Índice Maestro Alfabético (A-Z Quick Lookup) — Sección 5

# 1. Introducción y Contexto Histórico

La versión **Alpha 1.2.6** de Minecraft (diciembre de 2010) concluyó la era de desarrollo Alpha previa al inicio de la fase Beta 1.0. Esta entrega consolidó las características de la recordada *Halloween Update*: la dimensión paralela del **Nether**, el sistema de biomas basados en mapas de ruido de temperatura y humedad, los bloques ignífugos de Netherrack y Glowstone, y los monstruos Ghasts y Hombres Cerdo Zombi.

El código fuente contenido en este proyecto fue decompilado con **Minecraft Coder Pack (MCP)** y comprende **457 clases**. Esta guía documenta cada script individual para permitir entender qué hace el motor y cómo modificarlo con precisión quirúrgica.

# 2. Arquitectura del Motor y Ciclos Fundamentales

### El Game Loop y Sincronización de Ticks (20 TPS)

El cliente orquesta su ejecución mediante `Minecraft.run()`. La simulación lógica avanza a **20 ticks por segundo fijos** (50 ms por ciclo) coordinada por `Timer.java`. El renderizado gráfico opera de manera desacoplada a la tasa máxima soportada por el monitor, interpolando posiciones mediante `renderPartialTicks` para garantizar movimientos ultra suaves.

### Espacio de Coordenadas, Chunks y Almacenamiento

El mundo se divide en columnas verticales de **16x16x128 bloques** ( `Chunk.java` ). Las coordenadas globales se trasladan a coordenadas de chunk mediante desplazamientos de bits ( `x >> 4` , `z >> 4` ). Cada chunk contiene un array plano de 32,768 bytes para IDs de bloque y arrays `NibbleArray` (4 bits empaquetados) para metadatos y niveles de iluminación.

### Tubería Gráfica y Renderizado Inmediato (Tessellator)

El cliente utiliza la función fija de OpenGL sobre **LWJGL**. `Tessellator.java` acumula vértices, normales, coordenadas UV y colores en memoria nativa directa (NIO ByteBuffers), disparando miles de caras poligonales a la tarjeta de video con una única invocación `glDrawArrays`. Para el mundo, `WorldRenderer.java` compila cubos de 16x16x16 en listas de visualización OpenGL (Display Lists) para acelerar los FPS.

### Protocolo Multijugador (SMP) y NBT

El multijugador opera sobre conexiones TCP asíncronas con hilos dedicados de lectura y escritura en `NetworkManager.java`. Los paquetes ( `Packet0KeepAlive` a `Packet255KickDisconnect` ) serializan el estado del juego. Por su parte, el árbol jerárquico `NBTTagCompound` almacena datos de inventario, entidades y mundos con compresión GZIP.

# 3. Guía Rápida de Modificación (Modder's CookBook)

### 1. Cómo añadir un Bloque Nuevo

```
 package net.minecraft.src;
import java.util.Random;

public class BlockRuby extends Block {
    public BlockRuby(int id, int tex) {
        super(id, tex, Material.rock);
        setHardness(3.0F); setResistance(5.0F);
        setStepSound(soundStoneFootstep); setLightValue(0.5F);
    }
    public int idDropped(int meta, Random rand) { return Item.diamond.shiftedIndex; }
}

// En Block.java:
public static final Block rubyBlock = (new BlockRuby(95, 120)).setBlockName("rubyBlock");
// En Item.java:
Item.itemsList[Block.rubyBlock.blockID] = new ItemBlock(Block.rubyBlock.blockID - 256);
```

### 2. Cómo añadir un Ítem Nuevo

```
 package net.minecraft.src;

public class ItemRubyWand extends Item {
    public ItemRubyWand(int id) {
        super(id); maxStackSize = 1; setMaxDamage(64); setIconIndex(50);
    }
    public ItemStack onItemRightClick(ItemStack stack, World world, EntityPlayer player) {
        stack.damageItem(1);
        world.playSoundAtEntity(player, "random.bow", 0.5F, 0.4F);
        if(!world.multiplayerWorld) world.entityJoinedWorld(new EntitySnowball(world, player));
        return stack;
    }
}
// En Item.java:
public static Item rubyWand = (new ItemRubyWand(400)).setItemName("rubyWand");
```

### 3. Cómo registrar Recetas de Crafteo y Horno

```
 // En CraftingManager.java:
addRecipe(new ItemStack(Item.rubyWand, 1), new Object[] {
    "  R", " S ", "S  ", 'R', Block.rubyBlock, 'S', Item.stick
});
// En FurnaceRecipes.java:
addSmelting(Block.rubyBlock.blockID, new ItemStack(Item.diamond, 1));
```

## 4. Catálogo Exhaustivo de los 13 Subsistemas (457 Clases)

# 01. Núcleo y Ciclo del Juego (Core Game Loop & Engine) (34 clases)

> Clases fundamentales que inicializan la máquina virtual, gestionan la ventana de visualización mediante LWJGL, coordinan el ciclo de ticks (20 TPS) frente a la tasa de cuadros (FPS), procesan la entrada del usuario y manejan las sesiones y excepciones críticas.

## CanvasCrashReport

`class extends Canvas • 17 líneas`

**Lienzo de dibujo para la pantalla de reporte de fallo (Crash Report).**

Dibuja el texto del error y la traza de pila de la excepción si Minecraft se detiene inesperadamente.

> **Modding:** Permite personalizar el aspecto de la pantalla de error.

## CanvasIsomPreview

`class extends Canvas implements KeyListener, MouseListener, MouseMotionListener, Runnable • 466 líneas`

**Lienzo de dibujo del visor de mapas isométrico integrado en Alpha.**

Renderiza la proyección isométrica del mundo en tiempo real.

> **Modding:** Permite modificar la escala y resolución del visor isométrico.

Métodos Clave:

- `File getDataFolder()`
- `File func_1264_a(String s)`
- `void func_1270_b(String s)`
- `void func_1272_b()`
- `void func_1273_c()`

## CanvasMajongLogo

`class extends Canvas • 33 líneas`

**Lienzo gráfico para dibujar el logotipo inicial de Mojang.**

Muestra el logo corporativo durante la carga inicial del cliente.

> **Modding:** Permite personalizar el logotipo de arranque de un modpack.

Métodos Clave:

- `void paint(Graphics g)`

## CanvasMinecraftApplet

`class extends Canvas • 30 líneas`

**Componente Canvas de AWT para el lienzo de renderizado del Applet.**

Controla el repintado y dimensiones del área gráfica dentro del navegador.

> **Modding:** Modificar si se ajusta la integración con la ventana AWT.

## EnumOS1

`enum • 49 líneas`

**Enumeración de sistemas operativos compatibles (Windows, macOS, Solaris, Linux).**

Utilizada en la detección del entorno de ejecución.

> **Modding:** Parte del subsistema de compatibilidad multiplataforma.

Métodos Clave:

- `EnumOS1[] func_1064_values()`
- `EnumOS1 valueOf(String s)`

## EnumOS2

`enum • 50 líneas`

**Enumeración auxiliar de variantes de sistemas operativos.**

Complementa EnumOS1 para la clasificación detallada de la plataforma.

> **Modding:** Parte del subsistema de detección del entorno.

Métodos Clave:

- `EnumOS2[] values()`
- `EnumOS2 valueOf(String s)`

## EnumOSMappingHelper

`class • 37 líneas`

**Ayudante de mapeo entre cadenas de arquitectura y valores de enumeración de SO.**

Normaliza las propiedades de sistema 'os.name' de Java.

> **Modding:** Garantiza la portabilidad de rutas entre Windows, Linux y Mac.

## GameSettings

`class • 391 líneas`

**Gestor del archivo de configuración options.txt y preferencias del usuario.**

Serializa y almacena opciones gráficas, distancia de renderizado, FOV, volumen y enlaces de teclas (KeyBinding).

> **Modding:** Añade nuevas opciones para mods registrándolas en esta clase.

Métodos Clave:

- `String getKeyBinding(int i)`
- `void setKeyBinding(int i, int j)`
- `void setOptionFloatValue(int i, float f)`
- `void setOptionValue(int i, int j)`
- `int getOptionControlType(int i)`

## GameWindowListener

`class extends WindowAdapter • 35 líneas`

**Oyente de eventos de la ventana gráfica de AWT/LWJGL.**

Detecta el cierre de la ventana para guardar el mundo y pausar la partida si se pierde el foco.

> **Modding:** Permite interceptar el evento de apagado para liberar recursos personalizados.

Métodos Clave:

- `void windowClosing(WindowEvent windowevent)`

## KeyBinding

`class • 18 líneas`

**Asociación entre una acción de juego y una tecla física del teclado.**

Almacena el código de tecla de LWJGL, la descripción textual y el estado actual de pulsación.

> **Modding:** Crear nuevas instancias de KeyBinding para añadir teclas de acceso rápido a mods.

## Minecraft

`class implements Runnable • 1492 líneas`

**El orquestador maestro y corazón del cliente de Minecraft.**

Inicializa la máquina virtual, crea la ventana LWJGL Display, gestiona el bucle principal de ejecución 'while(running)', coordina el reloj de ticks a 20 TPS fijos mediante Timer, maneja la carga de texturas, sonido, entrada de teclado y ratón, cambios de pantalla GUI (displayGuiScreen) y renderizado general.

> **Modding:** La clase más importante del cliente: punto de partida para inyectar inicializaciones de mods o ganchos globales.

Métodos Clave:

- `void func_6258_a(String s, int i)`
- `void startGame()`
- `void func_6274_a(int i, int j, int k, int l, int i1, int j1)`
- `File getMinecraftDir()`
- `File getAppDir(String s)`

## MinecraftApplet

`class extends Applet • 145 líneas`

**Punto de entrada para ejecutar Minecraft dentro de un navegador web mediante Java Applet.**

Inicializa el contenedor Canvas de AWT y arranca el hilo principal de ejecución delegando en MinecraftAppletImpl.

> **Modding:** Relevante si se desea recrear o emular la ejecución web clásica de Minecraft en navegadores antiguos.

Métodos Clave:

- `void init()`
- `void func_6233_a()`
- `void start()`
- `void stop()`
- `void destroy()`

## MinecraftAppletImpl

`class extends Minecraft • 28 líneas`

**Implementación concreta de Minecraft asociada a la ventana Canvas del Applet.**

Sobrescribe métodos de resolución de pantalla y manejo de errores para adaptar el juego al marco del navegador.

> **Modding:** Maneja la integración entre el ciclo de vida del Applet y el bucle principal de Minecraft.

Métodos Clave:

- `void func_4007_a(UnexpectedThrowable unexpectedthrowable)`

## MinecraftError

`class extends Error • 13 líneas`

**Error irrecuperable lanzado por el motor cuando ocurre un fallo crítico de Java.**

Indica que el hilo de ejecución debe detenerse de inmediato para evitar corrupción del mundo.

> **Modding:** Capturado por el gestor de crashes del juego.

## MinecraftException

`class extends RuntimeException • 14 líneas`

**Excepción general del juego para fallos de carga o guardado.**

Lanzada ante errores de lectura de chunks o recursos.

> **Modding:** Útil para manejar errores controlados en lógica de mods.

## MinecraftImpl

`class extends Minecraft • 27 líneas`

**Implementación estándar de Minecraft para ejecución nativa en escritorio.**

Adapta la creación del marco principal y manejo de fallos para entornos de escritorio autónomos.

> **Modding:** Punto de entrada alternativo al cliente standalone.

Métodos Clave:

- `void func_4007_a(UnexpectedThrowable unexpectedthrowable)`

## MouseHelper

`class • 57 líneas`

**Controlador de bajo nivel para el puntero del ratón.**

Captura y centra el cursor dentro de la ventana de juego para permitir el movimiento continuo de cámara.

> **Modding:** Modificar si se implementa control táctil o sensibilidad no lineal.

Métodos Clave:

- `void func_774_a()`
- `void func_773_b()`
- `void func_772_c()`

## MovementInput

`class • 36 líneas`

**Clase base de entrada de movimiento para el jugador.**

Almacena las variables cinemáticas moveStrafe (lateral) y moveForward (adelante/atrás), y los estados de salto y agachado.

> **Modding:** Heredar de esta clase para crear sistemas de control por IA, mando (gamepad) o cinemáticas automáticas.

Métodos Clave:

- `void func_797_a(EntityPlayer entityplayer)`
- `void func_798_a()`
- `void func_796_a(int i, boolean flag)`

## MovementInputFromOptions

`class extends MovementInput • 89 líneas`

**Lee el estado físico del teclado según las opciones configuradas en GameSettings.**

Transforma la pulsación de WASD, espacio y shift en vectores de movimiento para EntityPlayerSP.

> **Modding:** Permite remapear o invertir controles.

Métodos Clave:

- `void func_796_a(int i, boolean flag)`
- `void func_798_a()`
- `void func_797_a(EntityPlayer entityplayer)`

## OsMap

`class • 36 líneas`

**Mapeo de opciones y rutas según el sistema operativo detectado.**

Proporciona la ruta canónica del directorio de datos (.minecraft / Application Support).

> **Modding:** Modificar si se desea reubicar el directorio raíz del juego.

## PanelCrashReport

`class extends Panel • 92 líneas`

**Panel contenedor AWT para la presentación de informes de errores.**

Muestra los detalles técnicos del fallo y botones para copiar el informe.

> **Modding:** Permite añadir información diagnóstica adicional a los informes de error.

## PlayerController

`class • 126 líneas`

**Clase base abstracta para controladores de modo de juego del jugador.**

Desacopla la interacción con bloques y mobs según las reglas del modo activo.

> **Modding:** Base de PlayerControllerSP y PlayerControllerMP.

Métodos Clave:

- `void func_717_a(World world)`
- `void clickBlock(int i, int j, int k, int l)`
- `boolean sendBlockRemoved(int i, int j, int k, int l)`
- `void sendBlockRemoving(int i, int j, int k, int l)`
- `void func_6468_a()`

## PlayerControllerMP

`class extends PlayerController • 212 líneas`

**Controlador del jugador para partidas multijugador.**

Empaqueta las acciones de rotura y colocación en paquetes Packet14BlockDig y Packet15Place para validación en el servidor.

> **Modding:** Modificar para implementar acciones predictivas o validaciones cliente en multijugador.

Métodos Clave:

- `void flipPlayer(EntityPlayer entityplayer)`
- `boolean sendBlockRemoved(int i, int j, int k, int l)`
- `void clickBlock(int i, int j, int k, int l)`
- `void func_6468_a()`
- `void sendBlockRemoving(int i, int j, int k, int l)`

## PlayerControllerSP

`class extends PlayerController • 147 líneas`

**Controlador del modo Supervivencia para un solo jugador.**

Gestiona los tiempos de rotura continua de bloques según herramienta, alcance del brazo y colocación inmediata.

> **Modding:** Modificar para alterar mecánicas de minado o combate en singleplayer.

Métodos Clave:

- `void flipPlayer(EntityPlayer entityplayer)`
- `boolean sendBlockRemoved(int i, int j, int k, int l)`
- `void clickBlock(int i, int j, int k, int l)`
- `void func_6468_a()`
- `void sendBlockRemoving(int i, int j, int k, int l)`

## PlayerControllerTest

`class extends PlayerController • 46 líneas`

**Controlador de pruebas experimentales y modo creativo preliminar.**

Permite rotura instantánea de bloques y recursos ilimitados para depuración interna.

> **Modding:** El precursor del modo creativo en Alpha.

Métodos Clave:

- `void func_6473_b(EntityPlayer entityplayer)`
- `boolean func_6469_d()`
- `void func_717_a(World world)`
- `void func_6474_c()`

## Session

`class • 57 líneas`

**Almacena las credenciales de la sesión activa del jugador.**

Contiene el nombre de usuario ('username') y el ID de sesión ('sessionId') validados ante Minecraft.net.

> **Modding:** Modificar si se desea cambiar de identidad en pruebas de desarrollo o implementar autenticación offline.

## ThreadConnectToServer

`class extends Thread • 63 líneas`

**Hilo asíncrono para negociar la conexión con un servidor multijugador.**

Abre el socket TCP sin congelar la pantalla de 'Connecting to server...'.

> **Modding:** Permite gestionar tiempos de espera (timeouts) en la conexión.

Métodos Clave:

- `void run()`

## ThreadDownloadImage

`class extends Thread • 55 líneas`

**Hilo secundario para descargar imágenes de texturas en segundo plano.**

Evita pausas en el renderizado mientras se obtienen imágenes remotas.

> **Modding:** Utilizado en la descarga de contenidos dinámicos.

Métodos Clave:

- `void run()`

## ThreadDownloadImageData

`class • 23 líneas`

**Contenedor y gestor de estado para imágenes descargadas asíncronamente.**

Supervisa la carga de bytes y notifica al renderizador cuando la textura está lista para subirse a la GPU.

> **Modding:** Maneja la textura por defecto mientras la descarga remota se completa.

## ThreadDownloadResources

`class extends Thread • 163 líneas`

**Hilo en segundo plano para descargar la música y efectos de sonido oficiales.**

Descarga canciones de C418 y efectos sonoros desde los servidores de recursos de Mojang en la carpeta .minecraft/resources.

> **Modding:** Controlar si se desean precargar sonidos locales en un mod.

Métodos Clave:

- `void run()`
- `void reloadResources()`
- `void closeMinecraft()`

## ThreadRunIsoClient

`class extends Thread • 29 líneas`

**Hilo de procesamiento para la generación del mapa isométrico.**

Coordina el renderizado continuo del visor ortogonal.

> **Modding:** Permite generar mapas en segundo plano.

Métodos Clave:

- `void run()`

## ThreadSleepForever

`class extends Thread • 32 líneas`

**Hilo utilitario que mantiene un hilo durmiendo indefinidamente con Thread.sleep().**

Utilizado como mecanismo de sincronización para mantener vivos hilos demonio en segundo plano.

> **Modding:** Utilidad interna de concurrencia.

Métodos Clave:

- `void run()`

## Timer

`class • 68 líneas`

**Sincronizador del reloj de juego (20 TPS) desacoplado de la tasa de cuadros (FPS).**

Mide los nanosegundos transcurridos para disparar exactamente 20 ticks lógicos por segundo mientras interpola 'renderPartialTicks' para que las animaciones a altos FPS sean fluidas.

> **Modding:** Ajusta timerSpeed para crear efectos de cámara lenta (slow-motion) o aceleración temporal.

Métodos Clave:

- `void updateTimer()`

## UnexpectedThrowable

`class • 18 líneas`

**Envoltorio estructurado para excepciones imprevistas.**

Almacena la descripción del fallo y el objeto Throwable para generar el volcado del Crash Report.

> **Modding:** Utilizado para reportar fallos de forma amigable.

# 02. Bloques y Materiales (Blocks, Materials & Mechanics) (66 clases)

El sistema de construcción y mundo voxel. Define los 256 IDs posibles de bloques, sus propiedades físicas (dureza, resistencia a explosiones, opacidad y emisión lumínica), mecánicas interactivas, gravedad, propagación de fluidos y redstone.

## Block

`class • 719 líneas`

**Clase base abstracta de todos los bloques en el mundo voxel.**

Define el registro global 'blocksList[256]', propiedades de opacidad, resistencia a explosiones, dureza de minado (blockHardness), emisión de luz y sonido. Gestiona el ciclo de vida (onBlockAdded, onBlockRemoval), ticks periódicos (updateTick), cajas AABB de colisión y selección, y drops de ítems.

> **Modding:** Para añadir un bloque nuevo, se instancia una subclase con un ID libre menor a 256 en blocksList. Para modificar la velocidad de rotura o drops globales, se tocan sus métodos base.

Métodos Clave:

- `Block setStepSound(StepSound stepsound)`
- `Block setLightOpacity(int i)`
- `Block setLightValue(float f)`
- `Block setResistance(float f)`
- `boolean renderAsNormalBlock()`

## BlockBloodStone

`class extends Block • 14 líneas`

**Netherrack / Piedra del Nether que mantiene el fuego perpetuo.**

El bloque geológico base del Inframundo. Tiene baja dureza de minado y la propiedad única de no extinguir jamás el fuego que arda en su cara superior.

> **Modding:** Ideal como plantilla para bloques con propiedades pirotécnicas o de dimensiones alternativas.

## BlockBookshelf

`class extends Block • 31 líneas`

**Manejador del bloque 'BlockBookshelf' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockBookshelf'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int getBlockTextureFromSide(int i)`
- `int quantityDropped(Random random)`

## BlockBreakable

`class extends Block • 34 líneas`

**Manejador del bloque 'BlockBreakable' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockBreakable'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `boolean isOpaqueCube()`
- `boolean shouldSideBeRendered(IBlockAccess iblockaccess, int i, int j, int k, int l)`

## BlockButton

`class extends Block • 329 líneas`

**Manejador del bloque 'BlockButton' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockButton'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `int tickRate()`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`

## BlockCactus

`class extends Block • 128 líneas`

**Manejador del bloque de cactus con daño por espinas y crecimiento sobre arena.**

Controla el comportamiento del cactus. Requiere arena inferior como apoyo y ningún bloque sólido vecino en las caras norte, sur, este u oeste. Crece verticalmente en updateTick hasta 3 bloques de altura e inflige 1 punto de daño en onEntityCollidedWithBlock a quien toque su caja reducida.

> **Modding:** Modifica onEntityCollidedWithBlock para alterar el daño de espinas, o canBlockStay() para permitir que crezca sobre tierra u otros materiales.

Métodos Clave:

- `void updateTick(World world, int i, int j, int k, Random random)`
- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `AxisAlignedBB getSelectedBoundingBoxFromPool(World world, int i, int j, int k)`
- `int getBlockTextureFromSide(int i)`
- `boolean renderAsNormalBlock()`

## BlockChest

`class extends BlockContainer • 273 líneas`

**Bloque de cofre para almacenamiento persistente de objetos.**

Hereda de BlockContainer y se asocia a TileEntityChest. Gestiona la detección de cofres contiguos para formar un 'cofre doble' de 54 casillas, bloquea la apertura si hay un bloque opaco encima, y expulsa todos los ítems contenidos al destruirse.

> **Modding:** Para crear cofres con mayor capacidad o con restricciones de acceso, se modifica esta clase junto con TileEntityChest y ContainerChest.

Métodos Clave:

- `int getBlockTexture(IBlockAccess iblockaccess, int i, int j, int k, int l)`
- `int getBlockTextureFromSide(int i)`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`
- `void onBlockRemoval(World world, int i, int j, int k)`
- `boolean blockActivated(World world, int i, int j, int k, EntityPlayer entityplayer)`

## BlockClay

`class extends Block • 25 líneas`

**Manejador del bloque 'BlockClay' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockClay'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int idDropped(int i, Random random)`
- `int quantityDropped(Random random)`

## BlockContainer

`class extends Block • 34 líneas`

**Manejador del bloque 'BlockContainer' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockContainer'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void onBlockAdded(World world, int i, int j, int k)`
- `void onBlockRemoval(World world, int i, int j, int k)`

## BlockCrops

`class extends BlockFlower • 138 líneas`

**Cultivo de trigo por etapas de crecimiento (de 0 a 7).**

Requiere tierra arada (BlockFarmland) inferior e hidratación. En updateTick, si la luz es >= 9, avanza sus metadatos hasta la etapa madura 7, donde suelta trigo y semillas.

> **Modding:** Ajusta la velocidad de crecimiento en updateTick o añade nuevos tipos de semillas y hortalizas.

Métodos Clave:

- `boolean canThisPlantGrowOnThisBlockID(int i)`
- `void updateTick(World world, int i, int j, int k, Random random)`
- `int getBlockTextureFromSideAndMetadata(int i, int j)`
- `int getRenderType()`
- `void onBlockDestroyedByPlayer(World world, int i, int j, int k, int l)`

## BlockDirt

`class extends Block • 14 líneas`

**Manejador del bloque 'BlockDirt' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockDirt'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

## BlockDoor

`class extends Block • 250 líneas`

**Mecánica de puertas de 2 bloques de altura con bisagras y apertura.**

Controla puertas de madera y hierro. Ocupa dos bloques verticales sincronizados mediante metadatos (mitad inferior y superior). Gestiona la apertura manual por clic o eléctrica por señal de redstone.

> **Modding:** Permite crear puertas de nuevos materiales ajustando canBlockStay y onPoweredBlockChange.

Métodos Clave:

- `int getBlockTextureFromSideAndMetadata(int i, int j)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`
- `AxisAlignedBB getSelectedBoundingBoxFromPool(World world, int i, int j, int k)`

## BlockFence

`class extends Block • 50 líneas`

**Manejador del bloque 'BlockFence' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockFence'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void getCollidingBoundingBoxes(World world, int i, int j, int k, AxisAlignedBB axisalignedbb, ArrayList arraylist)`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`

## BlockFire

`class extends Block • 317 líneas`

**Sistema de propagación de fuego, combustión y extinción.**

Controla la física destructiva del fuego. Mantiene tablas de inflamabilidad (chanceToEncourageFire) y combustibilidad (abilityToCatchFire). En updateTick propaga llamas a vecinos combustibles, calcula la extinción por lluvia o falta de combustible, y arde indefinidamente si la base es bloodStone (Netherrack).

> **Modding:** Añade nuevos bloques a setBurnRate() para hacerlos inflamables, o ajusta la velocidad de propagación en updateTick().

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`
- `int quantityDropped(Random random)`

## BlockFlower

`class extends Block • 74 líneas`

**Manejador del bloque 'BlockFlower' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockFlower'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `boolean canPlaceBlockAt(World world, int i, int j, int k)`
- `boolean canThisPlantGrowOnThisBlockID(int i)`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `void updateTick(World world, int i, int j, int k, Random random)`
- `void func_268_h(World world, int i, int j, int k)`

## BlockFlowing

`class extends BlockFluids • 322 líneas`

**Manejador del bloque 'BlockFlowing' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockFlowing'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void updateTick(World world, int i, int j, int k, Random random)`
- `int func_296_f(World world, int i, int j, int k, int l)`
- `void onBlockAdded(World world, int i, int j, int k)`

## BlockFluids

`class extends Block • 349 líneas`

**Manejador del bloque 'BlockFluids' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockFluids'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `float func_288_b(int i)`
- `int getBlockTextureFromSide(int i)`
- `int func_290_h(World world, int i, int j, int k)`
- `int func_289_b(IBlockAccess iblockaccess, int i, int j, int k)`
- `boolean renderAsNormalBlock()`

## BlockFurnace

`class extends BlockContainer • 181 líneas`

**Bloque de horno para fundición de minerales y cocinado de comida.**

Representa tanto el horno inactivo como el activo con fuego (usando dos IDs de bloque distintos para alternar la textura iluminada frontal). Es un BlockContainer asociado a TileEntityFurnace. Al interactuar con él (blockActivated), abre la interfaz de fundición y suelta sus contenidos al destruirse.

> **Modding:** Para cambiar la interfaz o la rotación al colocarse, se modifican blockActivated y onBlockPlaced.

Métodos Clave:

- `int idDropped(int i, Random random)`
- `void onBlockAdded(World world, int i, int j, int k)`
- `int getBlockTexture(IBlockAccess iblockaccess, int i, int j, int k, int l)`
- `void randomDisplayTick(World world, int i, int j, int k, Random random)`
- `int getBlockTextureFromSide(int i)`

## BlockGlass

`class extends BlockBreakable • 20 líneas`

**Manejador del bloque 'BlockGlass' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockGlass'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int quantityDropped(Random random)`

## BlockGrass

`class extends Block • 66 líneas`

**Bloque de césped con propagación y degradación a tierra.**

Define el césped superior del terreno. En updateTick, si la luz cae por debajo de 4 y tiene un bloque opaco encima, se transforma en tierra (Block.dirt). Si la luz es suficiente (>= 9), se propaga aleatoriamente a bloques de tierra adyacentes. Modula su color con ColorizerGrass.

> **Modding:** Modifica updateTick para cambiar la velocidad de propagación o getBlockTexture para personalizar su textura lateral/superior.

Métodos Clave:

- `int getBlockTexture(IBlockAccess iblockaccess, int i, int j, int k, int l)`
- `int colorMultiplier(IBlockAccess iblockaccess, int i, int j, int k)`
- `void updateTick(World world, int i, int j, int k, Random random)`
- `int idDropped(int i, Random random)`

## BlockGravel

`class extends BlockSand • 26 líneas`

**Manejador del bloque 'BlockGravel' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockGravel'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int idDropped(int i, Random random)`

## BlockIce

`class extends BlockBreakable • 50 líneas`

**Manejador del bloque 'BlockIce' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockIce'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int func_234_g()`
- `boolean shouldSideBeRendered(IBlockAccess iblockaccess, int i, int j, int k, int l)`
- `void onBlockRemoval(World world, int i, int j, int k)`
- `int quantityDropped(Random random)`
- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockJukeBox

`class extends Block • 60 líneas`

**Manejador del bloque 'BlockJukeBox' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockJukeBox'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int getBlockTextureFromSide(int i)`
- `boolean blockActivated(World world, int i, int j, int k, EntityPlayer entityplayer)`
- `void ejectRecord(World world, int i, int j, int k, int l)`
- `void dropBlockAsItemWithChance(World world, int i, int j, int k, int l, float f)`

## BlockLadder

`class extends Block • 148 líneas`

**Manejador del bloque 'BlockLadder' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockLadder'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `AxisAlignedBB getSelectedBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`

## BlockLeaves

`class extends BlockLeavesBase • 170 líneas`

**Bloque de hojas de árbol con descomposición por distancia al tronco.**

Gestiona las hojas de los árboles con soporte para gráficos 'Fancy' (transparentes) y 'Fast' (opacos). Comprueba si existe un bloque de madera (Block.wood) en un radio de 4 bloques; si no lo halla, decaen y tienen probabilidad de soltar retoños (Block.sapling).

> **Modding:** Ajusta la probabilidad de drop de saplings en idDropped/quantityDropped, o amplía la distancia de soporte para árboles colosales.

Métodos Clave:

- `int colorMultiplier(IBlockAccess iblockaccess, int i, int j, int k)`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `void func_6361_f(World world, int i, int j, int k, int l)`
- `void func_308_h(World world, int i, int j, int k)`
- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockLeavesBase

`class extends Block • 34 líneas`

**Manejador del bloque 'BlockLeavesBase' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockLeavesBase'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `boolean isOpaqueCube()`
- `boolean shouldSideBeRendered(IBlockAccess iblockaccess, int i, int j, int k, int l)`

## BlockLever

`class extends Block • 289 líneas`

**Manejador del bloque 'BlockLever' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockLever'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`

## BlockLightStone

`class extends Block • 20 líneas`

**Glowstone / Piedra luminosa del Nether.**

Bloque frágil que emite el nivel máximo de luz (15) y se genera colgado de los techos del Nether. Al destruirse suelta polvo de piedra brillante.

> **Modding:** Cambia idDropped y quantityDropped para alterar los recursos obtenidos al picarlo.

Métodos Clave:

- `int idDropped(int i, Random random)`

## BlockLog

`class extends Block • 35 líneas`

**Bloque de tronco / madera natural de árbol.**

Representa los troncos de madera sólida en el mundo. Proporciona madera talable que sirve de sustento a las hojas circundantes. Mapea la textura circular en las caras superior e inferior y la corteza en los laterales.

> **Modding:** Modifica getBlockTextureFromSide para añadir orientaciones de colocación o nuevos tipos de madera.

Métodos Clave:

- `int quantityDropped(Random random)`
- `int idDropped(int i, Random random)`
- `int getBlockTextureFromSide(int i)`

## BlockMinecartTrack

`class extends Block • 135 líneas`

**Manejador del bloque 'BlockMinecartTrack' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockMinecartTrack'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `MovingObjectPosition collisionRayTrace(World world, int i, int j, int k, Vec3D vec3d, Vec3D vec3d1)`
- `void setBlockBoundsBasedOnState(IBlockAccess iblockaccess, int i, int j, int k)`
- `int getBlockTextureFromSideAndMetadata(int i, int j)`

## BlockMobSpawner

`class extends BlockContainer • 35 líneas`

**Manejador del bloque 'BlockMobSpawner' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockMobSpawner'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `TileEntity SetBlockEntity()`
- `int idDropped(int i, Random random)`
- `int quantityDropped(Random random)`
- `boolean isOpaqueCube()`

## BlockMushroom

`class extends BlockFlower • 26 líneas`

**Manejador del bloque 'BlockMushroom' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockMushroom'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `boolean canThisPlantGrowOnThisBlockID(int i)`
- `boolean canBlockStay(World world, int i, int j, int k)`

## BlockObsidian

`class extends BlockStone • 25 líneas`

**Manejador del bloque 'BlockObsidian' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockObsidian'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int quantityDropped(Random random)`
- `int idDropped(int i, Random random)`

## BlockOre

`class extends Block • 35 líneas`

**Manejador del bloque 'BlockOre' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockOre'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int idDropped(int i, Random random)`
- `int quantityDropped(Random random)`

## BlockOreBlock

`class extends Block • 20 líneas`

**Manejador del bloque 'BlockOreBlock' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockOreBlock'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int getBlockTextureFromSide(int i)`

## BlockPortal

`class extends BlockBreakable • 207 líneas`

**Bloque de portal dimensional al Inframundo (Nether).**

Representa la lámina púrpura del portal dentro de un marco de obsidiana. Al colisionar una entidad con él, acumula tiempo para teletransportarla al Nether o de vuelta al Overworld usando Teleporter.

> **Modding:** Permite crear dimensiones adicionales personalizadas cambiando el teletransporte en onEntityCollidedWithBlock.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `void setBlockBoundsBasedOnState(IBlockAccess iblockaccess, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `boolean tryToCreatePortal(World world, int i, int j, int k)`

## BlockPressurePlate

`class extends Block • 194 líneas`

**Manejador del bloque 'BlockPressurePlate' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockPressurePlate'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int tickRate()`
- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`

## BlockPumpkin

`class extends Block • 91 líneas`

**Manejador del bloque 'BlockPumpkin' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockPumpkin'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int getBlockTextureFromSideAndMetadata(int i, int j)`
- `int getBlockTextureFromSide(int i)`
- `void onBlockAdded(World world, int i, int j, int k)`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`
- `void onBlockPlacedBy(World world, int i, int j, int k, EntityLiving entityliving)`

## BlockRedstoneOre

`class extends Block • 121 líneas`

**Manejador del bloque 'BlockRedstoneOre' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockRedstoneOre'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int tickRate()`
- `void onBlockClicked(World world, int i, int j, int k, EntityPlayer entityplayer)`
- `void onEntityWalking(World world, int i, int j, int k, Entity entity)`
- `boolean blockActivated(World world, int i, int j, int k, EntityPlayer entityplayer)`
- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockRedstoneTorch

`class extends BlockTorch • 224 líneas`

**Antorcha de redstone que actúa como inversor lógico NOT y fuente de energía.**

Alterna entre estado encendido y apagado. Cuando el bloque al que está anclada recibe energía, se apaga en el siguiente tick. Incluye un sistema contra burnout (quemado) si conmuta más de 8 veces en pocos ticks.

> **Modding:** Modifica updateTick para ajustar el retardo de conmutación o la sensibilidad al quemado.

Métodos Clave:

- `int getBlockTextureFromSideAndMetadata(int i, int j)`
- `int tickRate()`
- `void onBlockAdded(World world, int i, int j, int k)`
- `void onBlockRemoval(World world, int i, int j, int k)`
- `boolean isPoweringTo(IBlockAccess iblockaccess, int i, int j, int k, int l)`

## BlockRedstoneWire

`class extends Block • 402 líneas`

**Cable conductor de energía de redstone con atenuación de 15 a 0.**

El núcleo del sistema lógico de redstone. Transmite energía entre bloques adyacentes calculando la intensidad de señal (0 a 15) que se degrada 1 nivel por cada bloque de distancia. Notifica actualizaciones a los 6 bloques vecinos.

> **Modding:** Para modificar la distancia máxima de transmisión o la lógica de conexiones cruzadas, se reescribe updateSurroundingWires.

Métodos Clave:

- `int getBlockTextureFromSideAndMetadata(int i, int j)`
- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`

## BlockReed

`class extends Block • 110 líneas`

**Manejador del bloque 'BlockReed' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockReed'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void updateTick(World world, int i, int j, int k, Random random)`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `void checkBlockCoordValid(World world, int i, int j, int k)`
- `boolean canBlockStay(World world, int i, int j, int k)`

## BlockSand

`class extends Block • 78 líneas`

**Manejador del bloque 'BlockSand' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSand'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void onBlockAdded(World world, int i, int j, int k)`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `void updateTick(World world, int i, int j, int k, Random random)`
- `int tickRate()`
- `boolean canFallBelow(World world, int i, int j, int k)`

## BlockSapling

`class extends BlockFlower • 42 líneas`

**Manejador del bloque 'BlockSapling' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSapling'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockSign

`class extends BlockContainer • 136 líneas`

**Manejador del bloque 'BlockSign' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSign'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `AxisAlignedBB getSelectedBoundingBoxFromPool(World world, int i, int j, int k)`
- `void setBlockBoundsBasedOnState(IBlockAccess iblockaccess, int i, int j, int k)`
- `int getRenderType()`
- `boolean renderAsNormalBlock()`

## BlockSlowSand

`class extends Block • 26 líneas`

**Soul Sand / Arena de almas del Nether con reducción de velocidad.**

Ralentiza a cualquier entidad que camine sobre ella reduciendo su caja de colisión a 0.875 de altura, provocando que el jugador se hunda ligeramente.

> **Modding:** Ajusta getCollisionBoundingBoxFromPool para aumentar o reducir la fricción y hundimiento.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `void onEntityCollidedWithBlock(World world, int i, int j, int k, Entity entity)`

## BlockSnow

`class extends Block • 110 líneas`

**Manejador del bloque 'BlockSnow' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSnow'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`

## BlockSnowBlock

`class extends Block • 35 líneas`

**Manejador del bloque 'BlockSnowBlock' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSnowBlock'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int idDropped(int i, Random random)`
- `int quantityDropped(Random random)`
- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockSoil

`class extends Block • 133 líneas`

**Manejador del bloque 'BlockSoil' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSoil'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getBlockTextureFromSideAndMetadata(int i, int j)`
- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockSponge

`class extends Block • 51 líneas`

**Manejador del bloque 'BlockSponge' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockSponge'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void onBlockAdded(World world, int i, int j, int k)`
- `void onBlockRemoval(World world, int i, int j, int k)`

## BlockStairs

`class extends Block • 233 líneas`

**Manejador del bloque 'BlockStairs' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockStairs'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void setBlockBoundsBasedOnState(IBlockAccess iblockaccess, int i, int j, int k)`
- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`

## BlockStationary

`class extends BlockFluids • 73 líneas`

**Manejador del bloque 'BlockStationary' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockStationary'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `void updateTick(World world, int i, int j, int k, Random random)`

## BlockStep

`class extends Block • 91 líneas`

**Manejador del bloque 'BlockStep' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockStep'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int getBlockTextureFromSide(int i)`
- `boolean isOpaqueCube()`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `void onBlockAdded(World world, int i, int j, int k)`
- `int idDropped(int i, Random random)`

## BlockStone

`class extends Block • 20 líneas`

**Manejador del bloque 'BlockStone' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockStone'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `int idDropped(int i, Random random)`

## BlockTNT

`class extends Block • 65 líneas`

**Bloque de dinamita activable por golpe o redstone.**

Al romperse con la mano en Alpha o al recibir corriente de redstone, se destruye y crea una entidad EntityTNTPrimed con mecha activa de 80 ticks.

> **Modding:** Modifica onBlockDestroyedByPlayer para evitar que explote al golpearlo o cambiar la potencia.

Métodos Clave:

- `int getBlockTextureFromSide(int i)`
- `void onNeighborBlockChange(World world, int i, int j, int k, int l)`
- `int quantityDropped(Random random)`
- `void onBlockDestroyedByExplosion(World world, int i, int j, int k)`
- `void onBlockDestroyedByPlayer(World world, int i, int j, int k, int l)`

## BlockTorch

`class extends Block • 225 líneas`

**Manejador del bloque 'BlockTorch' en el mundo voxel.**

Define el comportamiento físico, renderizado, drops y colisiones para el bloque 'BlockTorch'. Gestiona su interacción con bloques vecinos y su ciclo de vida en el mundo.

> **Modding:** Sobrescribe métodos de interacción (blockActivated) o drops (idDropped/quantityDropped) para modificarlo.

Métodos Clave:

- `AxisAlignedBB getCollisionBoundingBoxFromPool(World world, int i, int j, int k)`
- `boolean isOpaqueCube()`
- `boolean renderAsNormalBlock()`
- `int getRenderType()`
- `boolean canPlaceBlockAt(World world, int i, int j, int k)`

## BlockWorkbench

`class extends Block • 40 líneas`

**Mesa de trabajo para crafteo en cuadrícula extendida de 3x3.**

Al hacer clic derecho (blockActivated), abre la pantalla GuiCrafting con el contenedor ContainerWorkbench de 9 casillas.

> **Modding:** Punto de referencia para crear mesas de artesanía especializadas (alquimia, herrería, magia).

Métodos Clave:

- `int getBlockTextureFromSide(int i)`
- `boolean blockActivated(World world, int i, int j, int k, EntityPlayer entityplayer)`

## Material

`class • 71 líneas`

**Definición de propiedades intrínsecas de la materia en el mundo de bloques.**

Indica si un material es inflamable, si requiere herramientas para dropear ítems, si es líquido o bloquea el movimiento.

> **Modding:** Añade constantes de material personalizadas para nuevos grupos de bloques con físicas afines.

Métodos Clave:

- `boolean getIsLiquid()`
- `boolean func_878_a()`
- `boolean getCanBlockGrass()`
- `boolean func_880_c()`
- `boolean getBurning()`

## MaterialLiquid

`class extends Material • 28 líneas`

**Especialización de Material para fluidos (agua y lava).**

Declara que el material es líquido y no bloquea el movimiento sólido pero sí ejerce resistencia al avance.

> **Modding:** Utilizado por bloques fluidos.

Métodos Clave:

- `boolean getIsLiquid()`
- `boolean func_880_c()`
- `boolean func_878_a()`

## MaterialLogic

`class extends Material • 28 líneas`

**Especialización de Material para componentes lógicos y circuitos.**

Define materiales que se destruyen inmediatamente al fluir agua sobre ellos (antorchas, redstone, repetidores).

> **Modding:** Usar para cables o circuitos que deban ser vulnerables a inundaciones.

Métodos Clave:

- `boolean func_878_a()`
- `boolean getCanBlockGrass()`
- `boolean func_880_c()`

## MaterialTransparent

`class extends Material • 28 líneas`

**Especialización de Material para bloques transparentes que no ocluyen luz.**

Utilizado por cristal, hielo y hojas de árboles.

> **Modding:** Usar para bloques translúcidos.

Métodos Clave:

- `boolean func_878_a()`
- `boolean getCanBlockGrass()`
- `boolean func_880_c()`

## MinecartTrackLogic

`class • 405 líneas`

**Lógica geométrica de conexión y curvatura de raíles de vagoneta.**

Calcula curvas, pendientes y bifurcaciones entre raíles contiguos basándose en las coordenadas y alturas de bloques vecinos.

> **Modding:** Crucial para implementar raíles propulsores, cambios de aguja o cruces complejos.

Métodos Clave:

- `void func_792_a(boolean flag)`

## RedstoneUpdateInfo

`class • 22 líneas`

**Registro de propagación para limitar actualizaciones cíclicas de redstone.**

Almacena coordenadas y timestamp para evitar bucles infinitos de encendido/apagado que congelen el juego.

> **Modding:** Protección crítica contra cierres inesperados por circuitos de reloj ultrarrápidos.

## StepSound

`class • 40 líneas`

**Efectos sonoros emitidos al caminar, colocar o destruir bloques.**

Asocia identificadores de audio a las acciones físicas realizadas sobre el bloque.

> **Modding:** Crea sonidos personalizados para bloques con acústica singular.

Métodos Clave:

- `float func_1147_b()`
- `float func_1144_c()`
- `String func_1146_a()`
- `String func_1145_d()`

## StepSoundSand

`class extends StepSound • 19 líneas`

**Variante de StepSound para arena, grava y nieve.**

Genera sonidos amortiguados y suaves al pisar o romper bloques granulares.

> **Modding:** Ajusta el tono y volumen para superficies arenosas.

Métodos Clave:

- `String func_1146_a()`

## StepSoundStone

`class extends StepSound • 19 líneas`

**Variante de StepSound para piedra, minerales y adoquín.**

Genera sonidos secos y rotundos característicos de la roca sólida.

> **Modding:** Ajusta el sonido de impacto en materiales pétreos.

Métodos Clave:

- `String func_1146_a()`

# 03. Objetos, Crafteo e Inventarios (Items, Crafting & Inventory) (47 clases)

Definición de herramientas, armas, armaduras, consumibles y materiales. Maneja la durabilidad, pilas (stacks), recetas en rejillas de 2x2 y 3x3 (CraftingManager), fundición en hornos y gestión de inventarios de jugador.

## CraftingInventoryCB

`class • 31 líneas`

**Contenedor abstracto base para interfaces con cuadrículas de crafteo.**

Gestiona la transferencia de ítems entre el inventario del jugador y la rejilla de fabricación.

**Modding:** Base para crear mesas de crafteo con tamaños alternativos (4x4, 5x5).

Métodos Clave:

- `void onCraftGuiClosed(EntityPlayer entityplayer)`
- `void onCraftMatrixChanged(IInventory iinventory)`

## CraftingInventoryPlayerCB

`class extends CraftingInventoryCB • 57 líneas`

**Contenedor para la cuadrícula de crafteo básica de 2x2 en el inventario del jugador.**

Gestiona las 4 casillas de crafteo personal y el slot de salida cuando se abre el inventario (tecla E).

**Modding:** Modificar para alterar el inventario personal del jugador.

Métodos Clave:

- `void onCraftMatrixChanged(IInventory iinventory)`
- `void onCraftGuiClosed(EntityPlayer entityplayer)`

## CraftingInventoryWorkbenchCB

`class extends CraftingInventoryCB • 55 líneas`

**Contenedor para la mesa de trabajo de 3x3 (Workbench).**

Gestiona las 9 casillas de fabricación y la casilla de resultado de crafteo.

**Modding:** Modificar para añadir funciones especiales a la mesa de trabajo.

Métodos Clave:

- `void onCraftMatrixChanged(IInventory iinventory)`
- `void onCraftGuiClosed(EntityPlayer entityplayer)`

## CraftingManager

`class • 229 líneas`

**Gestor y registro central de recetas de crafteo del juego.**

Mantiene la lista 'recipes' con todas las combinaciones en cuadrículas 2x2 y 3x3. Implementa addRecipe y addShapelessRecipe.

**Modding:** Llama a CraftingManager.getInstance().addRecipe(...) durante la inicialización para registrar recetas de mods.

Métodos Clave:

- `CraftingManager getInstance()`
- `ItemStack craft(int ai[])`

## CraftingRecipe

`class • 85 líneas`

**Definición y validación de una receta de crafteo específica.**

Compara la disposición de ítems en la cuadrícula con el patrón configurado para generar el resultado.

> **Modding:** Modificar si se implementan recetas complejas con condiciones dinámicas.

Métodos Clave:

- `boolean matchRecipe(int ai[])`
- `ItemStack createResult(int ai[])`
- `int getRecipeSize()`

## EnumArt

`enum • 119 líneas`

**Enumeración de cuadros artísticos disponibles (pinturas decorativas).**

Define dimensiones en bloques (1x1, 2x1, 4x4, etc.) y coordenadas UV en 'art/kz.png' para cada cuadro.

> **Modding:** Añade nuevos valores a esta enumeración para incorporar cuadros y pósteres adicionales.

Métodos Clave:

- `EnumArt[] func_1105_values()`
- `EnumArt valueOf(String s)`

## GuiCrafting

`class extends GuiContainer • 65 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiCrafting'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiCrafting'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void onGuiClosed()`
- `void drawGuiContainerForegroundLayer()`
- `void drawGuiContainerBackgroundLayer(float f)`

## InventoryCraftResult

`class implements IInventory • 59 líneas`

**Inventario de 1 sola casilla que alberga el producto resultante del crafteo.**

Se actualiza automáticamente cada vez que los ítems en la rejilla de fabricación forman una receta válida.

> **Modding:** Maneja la entrega del ítem final al jugador cuando hace clic.

Métodos Clave:

- `int getSizeInventory()`
- `ItemStack getStackInSlot(int i)`
- `String getInvName()`
- `ItemStack decrStackSize(int i, int j)`
- `void setInventorySlotContents(int i, ItemStack itemstack)`

## InventoryCrafting

`class implements IInventory • 82 líneas`

**Inventario de entrada para la cuadrícula de fabricación (2x2 o 3x3).**

Almacena los ítems colocados por el jugador y notifica al contenedor para recalcular la receta en tiempo real.

> **Modding:** Maneja la disposición espacial de los ingredientes.

Métodos Clave:

- `int getSizeInventory()`
- `ItemStack getStackInSlot(int i)`
- `String getInvName()`
- `ItemStack decrStackSize(int i, int j)`
- `void setInventorySlotContents(int i, ItemStack itemstack)`

## InventoryLargeChest

`class implements IInventory • 75 líneas`

**Contenedor virtual que fusiona dos TileEntityChest en un cofre doble de 54 casillas.**

Redirige las operaciones a uno u otro cofre según el índice del slot solicitado.

> **Modding:** Permite tratar dos inventarios independientes como uno solo.

Métodos Clave:

- `int getSizeInventory()`
- `String getInvName()`
- `ItemStack getStackInSlot(int i)`
- `ItemStack decrStackSize(int i, int j)`
- `void setInventorySlotContents(int i, ItemStack itemstack)`

## InventoryPlayer

`class implements IInventory • 502 líneas`

**El inventario completo del jugador (44 casillas en total).**

Gestiona las 36 casillas de almacenamiento, 4 de armadura, 1 de cursor en mano, el slot seleccionado en la hotbar y el descarte de ítems.

> **Modding:** Clase central para manipular el inventario del usuario desde el código.

Métodos Clave:

- `ItemStack getCurrentItem()`
- `void setCurrentItem(int i, boolean flag)`
- `void changeCurrentItem(int i)`
- `void decrementAnimations()`
- `boolean consumeInventoryItem(int i)`

## Item

`class • 213 líneas`

**Clase base de todos los objetos, herramientas y consumibles del juego.**

Define el registro global 'itemsList[32000]'. Los IDs de ítems arrancan en el índice 256 para no colisionar con los bloques. Define durabilidad (maxDamage), tamaño de pila (maxStackSize), textura en /gui/items.png (iconIndex) e interacciones onItemRightClick, onItemUse, hitEntity y onBlockDestroyed.

> **Modding:** Para crear un nuevo ítem, se instancia una subclase asignándole un ID >= 256 configurando sus propiedades.

Métodos Clave:

- `Item setIconIndex(int i)`
- `int getIconIndex(ItemStack itemstack)`
- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`
- `float getStrVsBlock(ItemStack itemstack, Block block)`
- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemArmor

`class extends Item • 32 líneas`

**Piezas de armadura corporal (casco, peto, pantalones, botas).**

Proporciona puntos de reducción de daño (damageReduceAmount), define el slot de armadura que ocupa (0 a 3) y la textura asignada al modelo del jugador.

> **Modding:** Modifica damageReduceAmount o crea conjuntos de armaduras de nuevos materiales.

## ItemAxe

`class extends ItemTool • 23 líneas`

**Ítem especializado 'ItemAxe' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemAxe'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

## ItemBlock

`class extends Item • 68 líneas`

**Ítem especializado 'ItemBlock' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemBlock'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemBoat

`class extends Item • 52 líneas`

**Ítem especializado 'ItemBoat' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemBoat'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemBow

`class extends Item • 29 líneas`

**Arco para disparo balístico de flechas.**

En onItemRightClick comprueba si el inventario del jugador contiene flechas (Item.arrow); si es así, consume una y genera la entidad EntityArrow con impulso parabólico.

> **Modding:** Modifica la velocidad del proyectil o implementa arcos con munición mágica o explosiva.

Métodos Clave:

- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemBucket

`class extends Item • 119 líneas`

**Ítem especializado 'ItemBucket' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemBucket'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemDoor

`class extends Item • 83 líneas`

**Ítem especializado 'ItemDoor' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemDoor'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemFishingRod

`class extends Item • 45 líneas`

**Ítem especializado 'ItemFishingRod' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemFishingRod'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean isFull3D()`
- `boolean shouldRotateAroundWhenRendering()`
- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemFlintAndSteel

`class extends Item • 53 líneas`

**Ítem especializado 'ItemFlintAndSteel' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemFlintAndSteel'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemFood

`class extends Item • 25 líneas`

**Alimento consumible que restaura salud directamente.**

En Alpha 1.2.6 no existía barra de hambre: la comida cura directamente corazones de vida (healAmount) al consumirse con clic derecho.

> **Modding:** Crea nuevos alimentos ajustando healAmount o implementando efectos secundarios.

Métodos Clave:

- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemHoe

`class extends Item • 58 líneas`

**Ítem especializado 'ItemHoe' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemHoe'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`
- `boolean isFull3D()`

## ItemMinecart

`class extends Item • 35 líneas`

**Ítem especializado 'ItemMinecart' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemMinecart'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemPainting

`class extends Item • 47 líneas`

**Ítem especializado 'ItemPainting' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemPainting'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemPickaxe

`class extends ItemTool • 55 líneas`

**Herramienta de pico para minar piedra y minerales.**

Aumenta drásticamente la velocidad al picar bloques de roca y permite recolectar minerales duros que con la mano no dropean nada.

> **Modding:** Añade nuevos bloques de piedra al array de eficacia para que este pico pueda extraerlos.

Métodos Clave:

- `boolean canHarvestBlock(Block block)`

## ItemRecord

`class extends Item • 32 líneas`

**Ítem especializado 'ItemRecord' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemRecord'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemRedstone

`class extends Item • 52 líneas`

**Ítem especializado 'ItemRedstone' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemRedstone'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemReed

`class extends Item • 66 líneas`

**Ítem especializado 'ItemReed' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemReed'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemRenderer

`class • 364 líneas`

**Ítem especializado 'ItemRenderer' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemRenderer'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `void renderItem(ItemStack itemstack)`
- `void renderItemInFirstPerson(float f)`
- `void renderOverlays(float f)`
- `void func_895_a()`
- `void func_9449_b()`

## ItemSaddle

`class extends Item • 34 líneas`

**Ítem especializado 'ItemSaddle' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemSaddle'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `void func_4019_b(ItemStack itemstack, EntityLiving entityliving)`
- `void hitEntity(ItemStack itemstack, EntityLiving entityliving)`

## ItemSeeds

`class extends Item • 35 líneas`

**Ítem especializado 'ItemSeeds' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemSeeds'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemSign

`class extends Item • 66 líneas`

**Ítem especializado 'ItemSign' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemSign'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean onItemUse(ItemStack itemstack, EntityPlayer entityplayer, World world, int i, int j, int k, int l)`

## ItemSnowball

`class extends Item • 27 líneas`

**Ítem especializado 'ItemSnowball' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemSnowball'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemSoup

`class extends ItemFood • 20 líneas`

**Ítem especializado 'ItemSoup' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemSoup'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `ItemStack onItemRightClick(ItemStack itemstack, World world, EntityPlayer entityplayer)`

## ItemSpade

`class extends ItemTool • 32 líneas`

**Ítem especializado 'ItemSpade' en el sistema de inventario.**

Define las propiedades (durabilidad, stack máximo, icono) e interacción al usar o equipar 'ItemSpade'. Registrado en el índice global de ítems.

> **Modding:** Sobrescribe onItemRightClick u onItemUse para dotarlo de efectos o habilidades personalizadas.

Métodos Clave:

- `boolean canHarvestBlock(Block block)`

## ItemStack

`class • 164 líneas`

**Instancia concreta de un ítem con cantidad y metadato/daño.**

Representa la unidad básica de almacenamiento en inventarios. Contiene itemID, stackSize (cantidad actual) e itemDamage (durabilidad consumida o subtipo). Gestiona el desgaste por uso (damageItem), división de pilas y serialización NBT.

> **Modding:** Clase crítica: cualquier manipulación de casillas de inventario pasa por crear o modificar instancias de ItemStack.

Métodos Clave:

- `ItemStack splitStack(int i)`
- `Item getItem()`
- `int getIconIndex()`
- `boolean useItem(EntityPlayer entityplayer, World world, int i, int j, int k, int l)`
- `float getStrVsBlock(Block block)`

## ItemSword

`class extends Item • 48 líneas`

**Arma de combate cuerpo a cuerpo con bonificación de daño.**

Calcula el daño adicional infligido a las entidades y sufre el doble de desgaste si se emplea para picar bloques sólidos.

> **Modding:** Ajusta weaponDamage para crear armas de mayor impacto o con efectos elementales.

Métodos Clave:

- `float getStrVsBlock(ItemStack itemstack, Block block)`
- `void hitEntity(ItemStack itemstack, EntityLiving entityliving)`
- `void hitBlock(ItemStack itemstack, int i, int j, int k, int l)`
- `int getDamageVsEntity(Entity entity)`
- `boolean isFull3D()`

## ItemTool

`class extends Item • 63 líneas`

**Clase base para herramientas de trabajo (picos, hachas, palas, azadas).**

Define la velocidad de minado frente a bloques específicos según el material (madera, piedra, hierro, diamante, oro). Aplica desgaste de durabilidad con cada bloque minado o entidad atacada.

> **Modding:** Modifica blocksEffectiveAgainst o el multiplicador de velocidad para balancear herramientas.

Métodos Clave:

- `float getStrVsBlock(ItemStack itemstack, Block block)`
- `void hitEntity(ItemStack itemstack, EntityLiving entityliving)`
- `void hitBlock(ItemStack itemstack, int i, int j, int k, int l)`
- `int getDamageVsEntity(Entity entity)`
- `boolean isFull3D()`

## RecipeSorter

`class implements Comparator • 32 líneas`

**Comparador que ordena las recetas en CraftingManager de mayor a menor complejidad.**

Asegura que recetas que ocupan 3x3 se evalúen antes que recetas que ocupan 2x2 para evitar colisiones erróneas.

> **Modding:** Crucial para mantener la precedencia correcta al registrar nuevas recetas.

Métodos Clave:

- `int func_1040_a(CraftingRecipe craftingrecipe, CraftingRecipe craftingrecipe1)`
- `int compare(Object obj, Object obj1)`

## RecipesArmor

`class • 56 líneas`

**Definición de recetas de piezas de armadura.**

Registra cascos, petos, pantalones y botas de cuero, hierro, oro y diamante.

> **Modding:** Referencia para registrar conjuntos de armaduras de nuevos materiales.

Métodos Clave:

- `void func_1148_a(CraftingManager craftingmanager)`

## RecipesCrafting

`class • 26 líneas`

**Definición de recetas básicas de construcción y utilidades.**

Registra recetas para bloques de construcción, antorchas, escaleras, cofres y mesas de trabajo.

> **Modding:** Ejemplo directo de sintaxis de recetas estándar.

Métodos Clave:

- `void func_1051_a(CraftingManager craftingmanager)`

## RecipesFood

`class • 23 líneas`

**Definición de recetas de alimentos compuestos.**

Registra la preparación de pan, pasteles, estofados de setas y galletas.

> **Modding:** Usar como plantilla para nuevas recetas culinarias.

Métodos Clave:

- `void func_976_a(CraftingManager craftingmanager)`

## RecipesIngots

`class • 40 líneas`

**Definición de recetas de compactación y descompactación de lingotes/bloques.**

Registra la conversión 9 lingotes <-> 1 bloque para hierro, oro, diamante y lapislázuli.

> **Modding:** Copiar para nuevos metales o minerales compactables.

Métodos Clave:

- `void func_810_a(CraftingManager craftingmanager)`

## RecipesTools

`class • 56 líneas`

**Definición de recetas de herramientas de trabajo.**

Registra picos, hachas, palas y azadas de todos los materiales.

> **Modding:** Referencia para registrar familias completas de herramientas.

Métodos Clave:

- `void func_1122_a(CraftingManager craftingmanager)`

## RecipesWeapons

`class • 50 líneas`

**Definición de recetas de espadas y armas.**

Registra las recetas de espadas de madera, piedra, hierro, diamante y oro.

> **Modding:** Modificar o copiar para registrar armas personalizadas.

Métodos Clave:

- `void func_766_a(CraftingManager craftingmanager)`

## SlotCrafting

`class extends SlotInventory • 34 líneas`

**Casilla especializada para el producto resultante de la fabricación.**

Al retirar el ítem de esta casilla, descuenta automáticamente los ingredientes de la cuadrícula de crafteo.

> **Modding:** Gestiona el consumo de materiales de fabricación.

Métodos Clave:

- `boolean isItemValid(ItemStack itemstack)`
- `void onPickupFromSlot()`

# 04. Entidades, Mobs y Físicas (Entities, Living Mobs, Vehicles & AI) (60 clases)

> Objetos dinámicos en el espacio 3D. Abarca al jugador, criaturas pasivas y hostiles, proyectiles (flechas, bolas de nieve), vehículos (vagonetas, barcas), entidades de bloques que caen (arena/grava), dinamita activa y el sistema de navegación por nodos A* (Pathfinder).

## Entity

`class • 1057 líneas`

**Clase base abstracta de todas las entidades y objetos físicos tridimensionales.**

La entidad raíz del motor. Gestiona posición (posX, posY, posZ), velocidad vectorial (motionX, motionY, motionZ), orientación (rotationYaw, rotationPitch), caja de colisión (AxisAlignedBB), estado en el suelo (onGround), fuego, inmersión en fluidos y persistencia NBT.

> **Modding:** Cualquier nuevo mob, proyectil o vehículo debe heredar directa o indirectamente de Entity e implementar writeToNBT/readFromNBT.

Métodos Clave:

- `boolean equals(Object obj)`
- `int hashCode()`
- `void preparePlayerToSpawn()`
- `void setEntityDead()`
- `void setSize(float f, float f1)`

## EntityAnimals

`class extends EntityCreature • 48 líneas`

**Entidad dinámica 'EntityAnimals' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityAnimals'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `float func_439_a(int i, int j, int k)`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `boolean getCanSpawnHere()`
- `int func_421_b()`

## EntityArrow

`class extends Entity • 287 líneas`

**Proyectil de flecha con física balística y anclaje en bloques.**

Calcula la trayectoria parabólica con gravedad y resistencia del aire. Realiza raycasting continuo; al impactar en una criatura le inflige daño, y al chocar contra un bloque se clava físicamente pudiendo ser recogida.

> **Modding:** Modifica la velocidad inicial, gravedad o añade efectos de impacto (explosión, fuego, teletransporte).

Métodos Clave:

- `void func_408_a(double d, double d1, double d2, float f, float f1)`
- `void setVelocity(double d, double d1, double d2)`
- `void onUpdate()`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`

## EntityBoat

`class extends Entity • 344 líneas`

**Entidad dinámica 'EntityBoat' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityBoat'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `AxisAlignedBB func_383_b_(Entity entity)`
- `AxisAlignedBB func_372_f_()`
- `boolean canBePushed()`
- `double func_402_h()`
- `boolean canAttackEntity(Entity entity, int i)`

## EntityBubbleFX

`class extends EntityFX • 46 líneas`

**Entidad dinámica 'EntityBubbleFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityBubbleFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void onUpdate()`

## EntityChicken

`class extends EntityAnimals • 97 líneas`

**Criatura pasiva de gallina que pone huevos y aletea al caer.**

Tiene caída lenta sin daño de caída aleteando sus alas. En cada intervalo de tiempo (timeUntilNextEgg) pone un huevo (Item.egg).

> **Modding:** Modifica la frecuencia de puesta de huevos o los drops cárnicos.

Métodos Clave:

- `void onLivingUpdate()`
- `void fall(float f)`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `String getLivingSound()`

## EntityClientPlayerMP

`class extends EntityPlayerSP • 183 líneas`

**Entidad dinámica 'EntityClientPlayerMP' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityClientPlayerMP'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean canAttackEntity(Entity entity, int i)`
- `void heal(int i)`
- `void onUpdate()`
- `void func_6420_o()`
- `void func_4056_N()`

## EntityCow

`class extends EntityAnimals • 67 líneas`

**Criatura pasiva de vaca ordeñable con cubos vacíos.**

Al hacer clic derecho con un cubo vacío (Item.bucketEmpty), llena el cubo con leche (Item.bucketMilk). Al morir suelta cuero.

> **Modding:** Modifica interact() para recolectar nuevos productos de la vaca.

Métodos Clave:

- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `String getLivingSound()`
- `String getHurtSound()`
- `String getDeathSound()`

## EntityCreature

`class extends EntityLiving • 170 líneas`

**Entidad dinámica 'EntityCreature' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityCreature'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_418_b_()`
- `void attackEntity(Entity entity, float f)`
- `float func_439_a(int i, int j, int k)`
- `Entity findPlayerToAttack()`
- `boolean getCanSpawnHere()`

## EntityCreeper

`class extends EntityMobs • 153 líneas`

**Mob hostil icónico: Creeper con detonación suicida por proximidad.**

Detecta al jugador en un radio cercano, silba mientras se hincha visualmente durante 30 ticks (timeSinceIgnited), y detona una explosión de potencia 3.0 al completar el temporizador. Si es alcanzado por un rayo, se sobrecarga.

> **Modding:** Modifica fuseTime o el radio de explosión para crear creepers con detonación modificada.

Métodos Clave:

- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `void func_9282_a(byte byte0)`
- `void onUpdate()`
- `void func_418_b_()`

## EntityDiggingFX

`class extends EntityFX • 60 líneas`

**Entidad dinámica 'EntityDiggingFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityDiggingFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `EntityDiggingFX func_4041_a(int i, int j, int k)`
- `int func_404_c()`
- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`

## EntityExplodeFX

`class extends EntityFX • 49 líneas`

**Entidad dinámica 'EntityExplodeFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityExplodeFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `void onUpdate()`

## EntityFX

`class extends Entity • 117 líneas`

**Entidad dinámica 'EntityFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `EntityFX func_407_b(float f)`
- `EntityFX func_405_d(float f)`
- `void onUpdate()`
- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `int func_404_c()`

## EntityFallingSand

`class extends Entity • 102 líneas`

**Entidad dinámica 'EntityFallingSand' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityFallingSand'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean canBeCollidedWith()`
- `void onUpdate()`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `float func_392_h_()`

## EntityFireball

`class extends Entity • 237 líneas`

**Entidad dinámica 'EntityFireball' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityFireball'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean func_384_a(double d)`
- `void onUpdate()`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `boolean canBeCollidedWith()`

## EntityFish

`class extends Entity • 390 líneas`

**Entidad dinámica 'EntityFish' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityFish'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean func_384_a(double d)`
- `void func_4042_a(double d, double d1, double d2, float f, float f1)`
- `void setPositionAndRotation2(double d, double d1, double d2, float f, float f1, int i)`
- `void setVelocity(double d, double d1, double d2)`
- `void onUpdate()`

## EntityFlameFX

`class extends EntityFX • 71 líneas`

**Entidad dinámica 'EntityFlameFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityFlameFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `float getEntityBrightness(float f)`
- `void onUpdate()`

## EntityFlying

`class extends EntityLiving • 81 líneas`

**Entidad dinámica 'EntityFlying' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityFlying'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void fall(float f)`
- `void func_435_b(float f, float f1)`
- `boolean isOnLadder()`

## EntityGhast

`class extends EntityFlying implements IMobs • 173 líneas`

**Mob hostil gigante volador del Inframundo (Nether).**

Vuela flotando por los espacios cavernosos del Nether, emite alaridos espectrales y dispara proyectiles ígneos explosivos cuando tiene línea de visión con el jugador.

> **Modding:** Ajusta la trayectoria de vuelo, la frecuencia de disparo o los proyectiles.

Métodos Clave:

- `void func_418_b_()`
- `String getLivingSound()`
- `String getHurtSound()`
- `String getDeathSound()`
- `int getDropItemId()`

## EntityItem

`class extends Entity • 219 líneas`

**Entidad dinámica 'EntityItem' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityItem'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void onUpdate()`
- `boolean handleWaterMovement()`
- `void func_355_a(int i)`
- `boolean canAttackEntity(Entity entity, int i)`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`

## EntityLavaFX

`class extends EntityFX • 65 líneas`

**Entidad dinámica 'EntityLavaFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityLavaFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `float getEntityBrightness(float f)`
- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `void onUpdate()`

## EntityList

`class • 143 líneas`

**Entidad dinámica 'EntityList' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityList'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `Entity createEntityInWorld(String s, World world)`
- `Entity createEntityFromNBT(NBTTagCompound nbttagcompound, World world)`
- `Entity createEntity(int i, World world)`
- `int getEntityID(Entity entity)`
- `String getEntityString(Entity entity)`

## EntityLiving

`class extends Entity • 867 líneas`

**Clase base para criaturas vivientes (con salud, respiración e IA).**

Extiende Entity añadiendo salud (health), aire bajo el agua, cálculo de daño recibido con invulnerabilidad temporal (hurtTime), animación de ataque (swingProgress), movimiento con inercia, salto y drops al morir en onDeath.

> **Modding:** Base fundamental para crear cualquier mob personalizado.

Métodos Clave:

- `boolean canEntityBeSeen(Entity entity)`
- `String getEntityTexture()`
- `boolean canBeCollidedWith()`
- `boolean canBePushed()`
- `float func_373_s()`

## EntityMinecart

`class extends Entity implements IInventory • 855 líneas`

**Entidad dinámica 'EntityMinecart' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityMinecart'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `AxisAlignedBB func_383_b_(Entity entity)`
- `AxisAlignedBB func_372_f_()`
- `boolean canBePushed()`
- `double func_402_h()`
- `boolean canAttackEntity(Entity entity, int i)`

## EntityMobs

`class extends EntityCreature implements IMobs • 109 líneas`

**Entidad dinámica 'EntityMobs' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityMobs'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void onLivingUpdate()`
- `void onUpdate()`
- `Entity findPlayerToAttack()`
- `boolean canAttackEntity(Entity entity, int i)`
- `void attackEntity(Entity entity, float f)`

## EntityOtherPlayerMP

`class extends EntityPlayer • 112 líneas`

**Entidad dinámica 'EntityOtherPlayerMP' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityOtherPlayerMP'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean canAttackEntity(Entity entity, int i)`
- `void setPositionAndRotation2(double d, double d1, double d2, float f, float f1, int i)`
- `void onUpdate()`
- `float func_392_h_()`
- `void onLivingUpdate()`

## EntityPainting

`class extends Entity • 238 líneas`

**Entidad dinámica 'EntityPainting' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityPainting'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_412_b(int i)`
- `void onUpdate()`
- `boolean func_410_i()`
- `boolean canBeCollidedWith()`
- `boolean canAttackEntity(Entity entity, int i)`

## EntityPickupFX

`class extends EntityFX • 65 líneas`

**Entidad dinámica 'EntityPickupFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityPickupFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `void onUpdate()`
- `int func_404_c()`

## EntityPig

`class extends EntityAnimals • 64 líneas`

**Criatura pasiva de cerdo montable con silla de montar.**

Camina pasivamente evitando precipicios. Si se le coloca una silla de montar (ItemSaddle), el jugador puede subirse sobre él.

> **Modding:** Modifica interact() para cambiar las condiciones de montura o doma.

Métodos Clave:

- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `String getLivingSound()`
- `String getHurtSound()`
- `String getDeathSound()`

## EntityPigZombie

`class extends EntityZombie • 126 líneas`

**Hombre cerdo zombi neutral del Nether con agresión colectiva en manada.**

Camina pasivamente por el Nether armado con una espada de oro. Si el jugador ataca a uno, todos los PigZombies en un amplio radio se vuelven inmediatamente hostiles.

> **Modding:** Estudia su sistema de venganza en manada para implementarlo en otras criaturas.

Métodos Clave:

- `void onUpdate()`
- `boolean getCanSpawnHere()`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `Entity findPlayerToAttack()`

## EntityPlayer

`class extends EntityLiving • 342 líneas`

**Representación del jugador dentro del mundo de Minecraft.**

Controla el inventario completo (InventoryPlayer), sistema de puntuación, interacción con bloques y mobs, capacidad de montar vehículos o cerdos, y cámara en primera y tercera persona.

> **Modding:** Punto de enganche para añadir mecánicas RPG, habilidades o inventarios extendidos.

Métodos Clave:

- `void func_350_p()`
- `void preparePlayerToSpawn()`
- `void func_418_b_()`
- `void onLivingUpdate()`
- `int func_6417_t()`

## EntityPlayerSP

`class extends EntityPlayer • 200 líneas`

**Instancia del jugador local en el cliente (procesa entrada de teclado/ratón).**

Conecta la entrada física del usuario (MovementInput) con la cinemática de EntityPlayer. En multijugador, empaqueta el movimiento en paquetes de red Packet10Flying.

> **Modding:** Permite modificar el control de movimiento, vuelo o interceptar comandos del cliente.

Métodos Clave:

- `void func_418_b_()`
- `void onLivingUpdate()`
- `void func_458_k()`
- `void func_460_a(int i, boolean flag)`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`

## EntityPortalFX

`class extends EntityFX • 72 líneas`

**Entidad dinámica 'EntityPortalFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityPortalFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `float getEntityBrightness(float f)`
- `void onUpdate()`

## EntityRainFX

`class extends EntityFX • 64 líneas`

**Entidad dinámica 'EntityRainFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityRainFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `void onUpdate()`

## EntityReddustFX

`class extends EntityFX • 74 líneas`

**Entidad dinámica 'EntityReddustFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityReddustFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `void onUpdate()`

## EntityRenderer

`class • 641 líneas`

**Entidad dinámica 'EntityRenderer' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityRenderer'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_911_a()`
- `void func_910_a(float f)`
- `void func_4136_b(float f)`
- `void func_4134_c(float f)`
- `void func_905_b()`

## EntitySheep

`class extends EntityAnimals • 65 líneas`

**Criatura pasiva de oveja con lana esquilable y tintada.**

Almacena en metadatos el color de su lana y el estado esquilado (sheared). Al ser atacada o esquilada con tijeras suelta bloques de lana (Block.cloth).

> **Modding:** Añade regeneración de lana al comer césped o nuevas paletas cromáticas.

Métodos Clave:

- `boolean canAttackEntity(Entity entity, int i)`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `String getLivingSound()`
- `String getHurtSound()`

## EntitySkeleton

`class extends EntityMobs • 93 líneas`

**Mob hostil a distancia armado con arco y flechas.**

Mantiene distancia con el jugador mientras dispara ráfagas de flechas (EntityArrow). También arde bajo la luz del sol diurna.

> **Modding:** Modifica la cadencia de disparo o la precisión de sus proyectiles.

Métodos Clave:

- `String getLivingSound()`
- `String getHurtSound()`
- `String getDeathSound()`
- `void onLivingUpdate()`
- `void attackEntity(Entity entity, float f)`

## EntitySlime

`class extends EntityLiving implements IMobs • 163 líneas`

**Entidad dinámica 'EntitySlime' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntitySlime'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_441_c(int i)`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `void onUpdate()`
- `void func_418_b_()`

## EntitySlimeFX

`class extends EntityFX • 42 líneas`

**Entidad dinámica 'EntitySlimeFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntitySlimeFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `int func_404_c()`
- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`

## EntitySmokeFX

`class extends EntityFX • 74 líneas`

**Entidad dinámica 'EntitySmokeFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntitySmokeFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `void func_406_a(Tessellator tessellator, float f, float f1, float f2, float f3, float f4, float f5)`
- `void onUpdate()`

## EntitySnowball

`class extends Entity • 270 líneas`

**Entidad dinámica 'EntitySnowball' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntitySnowball'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean func_384_a(double d)`
- `void func_467_a(double d, double d1, double d2, float f, float f1)`
- `void setVelocity(double d, double d1, double d2)`
- `void onUpdate()`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`

## EntitySorter

`class implements Comparator • 28 líneas`

**Entidad dinámica 'EntitySorter' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntitySorter'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `int func_1063_a(WorldRenderer worldrenderer, WorldRenderer worldrenderer1)`
- `int compare(Object obj, Object obj1)`

## EntitySpider

`class extends EntityMobs • 91 líneas`

**Mob hostil arácnido con capacidad para escalar paredes verticales.**

Detecta colisiones laterales con bloques para ascender verticalmente. En la oscuridad es agresiva, pero bajo la luz solar diurna se vuelve neutral a menos que sea atacada.

> **Modding:** Modifica findPlayerToAttack para alterar su agresividad según la luminosidad.

Métodos Clave:

- `double func_402_h()`
- `Entity findPlayerToAttack()`
- `String getLivingSound()`
- `String getHurtSound()`
- `String getDeathSound()`

## EntitySplashFX

`class extends EntityRainFX • 23 líneas`

**Entidad dinámica 'EntitySplashFX' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntitySplashFX'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

## EntityTNTPrimed

`class extends Entity • 87 líneas`

**Entidad dinámica 'EntityTNTPrimed' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityTNTPrimed'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `boolean canBeCollidedWith()`
- `void onUpdate()`
- `void writeEntityToNBT(NBTTagCompound nbttagcompound)`
- `void readEntityFromNBT(NBTTagCompound nbttagcompound)`
- `float func_392_h_()`

## EntityZombie

`class extends EntityMobs • 51 líneas`

**Mob hostil cuerpo a cuerpo que arde bajo la luz del sol.**

Persigue al jugador en línea recta mediante Pathfinder, ataca por contacto directo y entra en combustión si se encuentra expuesto a la luz solar diurna.

> **Modding:** Añade equipamiento, resistencia a la luz solar o drops especiales.

Métodos Clave:

- `void onLivingUpdate()`
- `String getLivingSound()`
- `String getHurtSound()`
- `String getDeathSound()`
- `int getDropItemId()`

## EntityZombieSimple

`class extends EntityMobs • 25 líneas`

**Entidad dinámica 'EntityZombieSimple' en el espacio 3D.**

Gestiona el movimiento, colisiones, renderizado y estados de vida para la entidad 'EntityZombieSimple'. Sincroniza su posición y datos mediante etiquetas NBT.

> **Modding:** Sobrescribe onUpdate() para modificar su comportamiento tick a tick o su IA.

Métodos Clave:

- `float func_439_a(int i, int j, int k)`

## EnumCreatureType

`enum • 45 líneas`

**Clasificación biológica de criaturas (Monstruos, Criaturas pasivas, Acuáticas).**

Establece el límite máximo de entidades simultáneas por chunk y el tipo de material donde pueden spawnear.

> **Modding:** Añade nuevas categorías ecológicas si desarrollas mods con fauna exótica.

Métodos Clave:

- `EnumCreatureType[] func_4145_values()`
- `EnumCreatureType valueOf(String s)`

## EnumMobType

`enum • 43 líneas`

**Clasificación del tipo de mob según comportamiento.**

Distingue entre entidades pasivas y hostiles para la asignación de objetivos.

> **Modding:** Utilizado en la lógica de IA y selección de objetivos.

Métodos Clave:

- `EnumMobType[] func_4131_values()`
- `EnumMobType valueOf(String s)`

## IMobs

`interface • 9 líneas`

**Interfaz marcadora que identifica a una entidad como monstruo hostil.**

Utilizada por el selector de objetivos de criaturas y por la dificultad para permitir o prohibir su spawn.

> **Modding:** Implementar en cualquier criatura personalizada que deba comportarse como monstruo.

## MobSpawnerBase

`class • 175 líneas`

**Clase base para la ecología de spawn de criaturas en biomas.**

Define las listas de monstruos agresivos, criaturas pacíficas y criaturas acuáticas que pueden aparecer.

> **Modding:** Punto de inyección para que nuevos mobs aparezcan naturalmente en el mundo.

Métodos Clave:

- `void generateBiomeLookup()`
- `MobSpawnerBase func_4122_b()`
- `MobSpawnerBase setBiomeName(String s)`
- `MobSpawnerBase func_4124_a(int i)`
- `MobSpawnerBase func_4123_b(int i)`

## MobSpawnerDesert

`class extends MobSpawnerBase • 13 líneas`

**Definición ecológica para el bioma de Desierto.**

Restringe la aparición de animales pasivos y ajusta los monstruos nocturnos.

> **Modding:** Ejemplo de especialización bioclimática.

## MobSpawnerHell

`class extends MobSpawnerBase • 17 líneas`

**Definición ecológica para el Inframundo (Nether).**

Configura la aparición exclusiva de Ghasts y PigZombies en el Inframundo.

> **Modding:** Añadir aquí nuevos mobs del Nether.

## MobSpawnerSwamp

`class extends MobSpawnerBase • 13 líneas`

**Definición ecológica para el bioma de Pantano.**

Configura las criaturas que pueden aparecer en ciénagas y humedales.

> **Modding:** Ejemplo de especialización bioclimática.

## Path

`class • 144 líneas`

**Encapsulador de una ruta de navegación calculada para criaturas.**

Almacena la secuencia de nodos PathPoint que un mob debe recorrer sucesivamente para alcanzar su destino.

> **Modding:** Utilizado por Pathfinder y EntityCreature.

Métodos Clave:

- `PathPoint addPoint(PathPoint pathpoint)`
- `void clearPath()`
- `PathPoint dequeue()`
- `void changeDistance(PathPoint pathpoint, float f)`
- `boolean isPathEmpty()`

## PathEntity

`class • 37 líneas`

**Secuencia ordenada de puntos de ruta (PathPoint) asignada a una entidad.**

Permite que un mob avance nodo a nodo comprobando si ha alcanzado el siguiente punto del camino.

> **Modding:** Utilizado en la navegación cinemática de criaturas vivas.

Métodos Clave:

- `void incrementPathIndex()`
- `boolean isFinished()`
- `Vec3D getPosition(Entity entity)`

## PathPoint

`class • 58 líneas`

**Nodo espacial tridimensional con coste de recorrido para el algoritmo A*.**

Almacena coordenadas enteras (x, y, z), índice en el heap binario, coste acumulado g y distancia estimada h al objetivo.

> **Modding:** El elemento básico de la búsqueda de caminos de Pathfinder.

Métodos Clave:

- `float distanceTo(PathPoint pathpoint)`
- `boolean equals(Object obj)`
- `int hashCode()`
- `boolean isAssigned()`
- `String toString()`

## Pathfinder

`class • 218 líneas`

**Motor de búsqueda de caminos sobre la malla de bloques navegables mediante algoritmo A*.**

Calcula la ruta óptima sorteando obstáculos, caídas y desniveles en el terreno para que los mobs alcancen al jugador.

> **Modding:** Fundamental para optimizar la inteligencia artificial o permitir que los mobs superen obstáculos complejos.

Métodos Clave:

- `PathEntity createEntityPathTo(Entity entity, Entity entity1, float f)`
- `PathEntity createEntityPathTo(Entity entity, int i, int j, int k, float f)`

## SpawnerAnimals

`class • 125 líneas`

**Motor biológico de spawn periódico de criaturas.**

Evalúa chunks en un radio alrededor del jugador en cada tick, comprueba el límite de población de bioma e instancia nuevos animales o monstruos.

> **Modding:** Punto clave para modificar la tasa de aparición o densidad ecológica del mundo.

Métodos Clave:

- `ChunkPosition getRandomSpawningPointInChunk(World world, int i, int j)`
- `int performSpawning(World var0)`

# 05. Generación de Mundo y Biomas (World Generation & Biomes) (19 clases)

> Algoritmos procedurales de generación de terreno mediante ruido Perlin 3D. Abarca el tallado de cuevas gusano (MapGenCaves), distribución de biomas (temperatura y humedad), y generadores de árboles, minerales, lagos, mazmorras y el Inframundo (Nether).

## MapGenBase

`class • 41 líneas`

**Clase base para generadores de estructuras geológicas del mapa.**

Procesa el terreno chunk a chunk aplicando transformaciones continuas (ej. tallado de cuevas).

> **Modding:** Extender para crear generadores de barrancos, fallas o túneles subterráneos.

Métodos Clave:

- `void func_867_a(IChunkProvider ichunkprovider, World world, int i, int j, byte abyte0[])`
- `void func_868_a(World world, int i, int j, int k, int l, byte abyte0[])`

## MapGenCaves

`class extends MapGenBase • 224 líneas`

**Algoritmo procedural de tallado de redes de cuevas subterráneas (worm caves).**

Genera túneles sinuosos que serpentean a través de la piedra creando cavernas, intersecciones y pozos usando ruido y trigonometría.

> **Modding:** Ajusta la densidad y amplitud de las cuevas para crear mundos más o menos cavernosos.

Métodos Clave:

- `void func_870_a(int i, int j, byte abyte0[], double d, double d1, double d2)`
- `void func_869_a(int i, int j, byte abyte0[], double d, double d1, double d2, float f, float f1, float f2, int k, int l, double d3)`
- `void func_868_a(World world, int i, int j, int k, int l, byte abyte0[])`

## MapGenCavesHell

`class extends MapGenBase • 209 líneas`

**Generador de cavernas y bolsas de lava específico del Inframundo (Nether).**

Talla cavidades redondeadas a gran escala dentro de la masa sólida de Netherrack.

> **Modding:** Modificar para alterar la morfología de las cavernas del Nether.

Métodos Clave:

- `void func_4129_a(int i, int j, byte abyte0[], double d, double d1, double d2)`
- `void func_4128_a(int i, int j, byte abyte0[], double d, double d1, double d2, float f, float f1, float f2, int k, int l, double d3)`
- `void func_868_a(World world, int i, int j, int k, int l, byte abyte0[])`

## WorldGenBigTree

`class extends WorldGenerator • 450 líneas`

**Generador de estructuras o recursos: 'WorldGenBigTree'.**

Inserta proceduralmente bloques en el terreno durante la fase de población de chunks para 'WorldGenBigTree'.

> **Modding:** Modifica las condiciones de colocación o el tipo de bloques generados.

Métodos Clave:

- `void func_517_a(double d, double d1, double d2)`
- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenCactus

`class extends WorldGenerator • 39 líneas`

**Generador superficial de columnas de cactus en biomas desérticos.**

Coloca cactus de 1 a 3 bloques de altura sobre bloques de arena que no tengan vecinos sólidos.

> **Modding:** Ajustar la frecuencia o altura de los cactus en desiertos.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenClay

`class extends WorldGenerator • 69 líneas`

**Generador de depósitos submarinos de arcilla.**

Genera vetas de arcilla en el lecho de lagos y playas poco profundas.

> **Modding:** Modificar para aumentar la disponibilidad de arcilla en el juego.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenDungeons

`class extends WorldGenerator • 214 líneas`

**Generador de estructuras o recursos: 'WorldGenDungeons'.**

Inserta proceduralmente bloques en el terreno durante la fase de población de chunks para 'WorldGenDungeons'.

> **Modding:** Modifica las condiciones de colocación o el tipo de bloques generados.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenFire

`class extends WorldGenerator • 30 líneas`

**Generador de fuegos eternos en el Inframundo (Nether).**

Coloca parches de fuego sobre superficies de Netherrack durante la generación del terreno.

> **Modding:** Controla la ambientación pirotécnica del Nether.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenFlowers

`class extends WorldGenerator • 33 líneas`

**Generador de parches de flores amarillas y rosas rojas.**

Genera agrupaciones de flores sobre bloques de hierba con exposición a la luz solar.

> **Modding:** Modificar para introducir nuevas especies botánicas en el paisaje.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenHellLava

`class extends WorldGenerator • 79 líneas`

**Generador de manantiales y cascadas de lava en los muros del Nether.**

Perfora cavidades en los acantilados de Netherrack colocando bloques de lava que caen en cascada hacia el suelo.

> **Modding:** Crea el paisaje característico de cataratas de lava del Inframundo.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenLakes

`class extends WorldGenerator • 115 líneas`

**Generador de lagos superficiales y subterráneos de agua o lava.**

Talla una cuenca redondeada en el terreno y la llena con el líquido seleccionado.

> **Modding:** Ajustar la probabilidad o líquido de los lagos generados.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenLightStone1

`class extends WorldGenerator • 77 líneas`

**Generador de estalactitas colgantes de piedra luminosa (Glowstone) en el techo del Nether.**

Genera racimos suspendidos de Glowstone que iluminan las cavernas superiores del Inframundo.

> **Modding:** Modificar para alterar la forma o frecuencia de la Glowstone en el Nether.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenLightStone2

`class extends WorldGenerator • 77 líneas`

**Segunda variante del generador de Glowstone en paredes y recovecos del Nether.**

Produce acumulaciones de piedra brillante incrustadas en salientes rocosos.

> **Modding:** Complementa WorldGenLightStone1.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenLiquids

`class extends WorldGenerator • 75 líneas`

**Generador de pequeños manantiales de agua o lava que brotan de las paredes de piedra.**

Perfora un bloque en una pared rocosa y coloca una fuente que fluye hacia el exterior.

> **Modding:** Crea manantiales naturales dentro de cuevas.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenMinable

`class extends WorldGenerator • 60 líneas`

**Generador de estructuras o recursos: 'WorldGenMinable'.**

Inserta proceduralmente bloques en el terreno durante la fase de población de chunks para 'WorldGenMinable'.

> **Modding:** Modifica las condiciones de colocación o el tipo de bloques generados.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenPumpkin

`class extends WorldGenerator • 30 líneas`

**Generador de calabazas silvestres en la superficie del Overworld.**

Genera grupos dispersos de calabazas talladas en zonas de hierba.

> **Modding:** Modificar para variar la rareza de las calabazas.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenReed

`class extends WorldGenerator • 39 líneas`

**Generador de cañas de azúcar en orillas de ríos y costas.**

Coloca cañas de azúcar de 1 a 3 bloques de altura sobre tierra o arena adyacente a un bloque de agua.

> **Modding:** Ajustar para cambiar la frecuencia de cañas en el mundo.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenTrees

`class extends WorldGenerator • 96 líneas`

**Generador de estructuras o recursos: 'WorldGenTrees'.**

Inserta proceduralmente bloques en el terreno durante la fase de población de chunks para 'WorldGenTrees'.

> **Modding:** Modifica las condiciones de colocación o el tipo de bloques generados.

Métodos Clave:

- `boolean generate(World world, Random random, int i, int j, int k)`

## WorldGenerator

`class • 20 líneas`

**Generador de estructuras o recursos: 'WorldGenerator'.**

Inserta proceduralmente bloques en el terreno durante la fase de población de chunks para 'WorldGenerator'.

> **Modding:** Modifica las condiciones de colocación o el tipo de bloques generados.

Métodos Clave:

- `void func_517_a(double d, double d1, double d2)`

# 06. Mundo, Chunks, Iluminación y Guardado (World, Chunks, Lighting & Storage) (30 clases)

Estructura espacial y de persistencia. El mundo dividido en Chunks de 16x16x128 bloques, arrays de bloques y metadatos de 4 bits (NibbleArray), cálculo y propagación de iluminación solar y de bloques, cola de ticks programados y serialización en disco.

## Chunk

`class • 691 líneas`

**Segmento vertical del terreno de 16x16x128 bloques.**

Almacena arrays planos para IDs de bloques (32768 bytes), metadatos (NibbleArray), luz solar y luz de bloque. Gestiona el mapa de alturas (heightMap) y entidades del área.

**Modding:** Optimiza o inspecciona esta clase para manipulaciones masivas de terreno de alto rendimiento.

Métodos Clave:

- `boolean isAtLocation(int i, int j)`
- `int getHeightValue(int i, int j)`
- `void func_1014_a()`
- `void generateHeightMap()`
- `void func_1024_c()`

## ChunkCache

`class implements IBlockAccess • 169 líneas`

**Caché ligera de chunks adyacentes para operaciones rápidas de renderizado.**

Implementa IBlockAccess sobre un rango acotado de chunks, evitando sincronizaciones pesadas con World.

**Modding:** Utilizada intensamente por RenderBlocks para consultar iluminación y vecinos sin penalización.

Métodos Clave:

- `int getBlockId(int i, int j, int k)`
- `TileEntity getBlockTileEntity(int i, int j, int k)`
- `float getLightBrightness(int i, int j, int k)`
- `int func_4086_d(int i, int j, int k)`
- `int func_716_a(int i, int j, int k, boolean flag)`

## ChunkCoordIntPair

`class • 29 líneas`

**Par inmutable de coordenadas enteras (chunkX, chunkZ) para indexación de chunks.**

Implementa hashCode() y equals() para servir como clave en tablas hash de chunks cargados.

**Modding:** Clase fundamental para referenciar chunks en mapas y conjuntos.

Métodos Clave:

- `int hashCode()`
- `boolean equals(Object obj)`

## ChunkCoordinates

`class • 35 líneas`

**Tupla de coordenadas espaciales tridimensionales de bloque (x, y, z).**

Almacena puntos en el espacio para puntos de spawn, camas o marcadores de brújula.

> **Modding:** Útil para guardar y comparar posiciones tridimensionales.

Métodos Clave:

- `boolean equals(Object obj)`
- `int hashCode()`

## ChunkLoader

`class implements IChunkLoader • 243 líneas`

**Implementación de almacenamiento de chunks en disco en formato clásico de Alpha.**

Serializa cada chunk en archivos binarios individuales organizados en subdirectorios calculados por base 64.

> **Modding:** Relevante si se diseñan conversores de mapas o guardado alternativo.

Métodos Clave:

- `Chunk loadChunk(World world, int i, int j)`
- `void saveChunk(World world, Chunk chunk)`
- `void storeChunkInCompound(Chunk chunk, World world, NBTTagCompound nbttagcompound)`
- `Chunk loadChunkIntoWorldFromCompound(World world, NBTTagCompound nbttagcompound)`
- `void func_814_a()`

## ChunkPosition

`class • 37 líneas`

**Contenedor ligero de posición tridimensional entera (x, y, z).**

Utilizado en cálculos de propagación de explosiones y algoritmos de búsqueda de caminos.

> **Modding:** Útil como estructura ligera de coordenadas inmutables.

Métodos Clave:

- `boolean equals(Object obj)`
- `int hashCode()`

## ChunkProviderClient

`class implements IChunkProvider • 86 líneas`

**Proveedor de chunks ficticio en el cliente multijugador.**

Almacena en memoria únicamente los chunks recibidos desde el servidor a través de paquetes de red.

> **Modding:** Punto clave para gestionar los chunks visibles en el cliente multijugador.

Métodos Clave:

- `boolean chunkExists(int i, int j)`
- `void func_539_c(int i, int j)`
- `Chunk func_538_d(int i, int j)`
- `Chunk provideChunk(int i, int j)`
- `boolean saveChunks(boolean flag, IProgressUpdate iprogressupdate)`

## ChunkProviderGenerate

`class implements IChunkProvider • 609 líneas`

**Generador procedural principal de terreno para el Overworld.**

Calcula la topografía mediante matrices de ruido Perlin 3D, genera capas de piedra, tierra, agua, y llama a 'populate' para árboles y minerales.

> **Modding:** El punto central para modificar la forma de las montañas, islas flotantes o continentes del mundo.

Métodos Clave:

- `void generateTerrain(int i, int j, byte abyte0[], MobSpawnerBase amobspawnerbase[], double ad[])`
- `void replaceBlocksForBiome(int i, int j, byte abyte0[], MobSpawnerBase amobspawnerbase[])`
- `Chunk provideChunk(int i, int j)`
- `boolean chunkExists(int i, int j)`
- `void populate(IChunkProvider ichunkprovider, int i, int j)`

## ChunkProviderHell

`class implements IChunkProvider • 419 líneas`

**Generador procedural del Inframundo (Nether).**

Genera una caverna infinita con techo y suelo de bedrock, lagos masivos de lava a Y=32, y pilares de netherrack.

> **Modding:** Modificar para alterar la generación geológica del Nether o crear dimensiones subterráneas.

Métodos Clave:

- `void func_4059_a(int i, int j, byte abyte0[])`
- `void func_4058_b(int i, int j, byte abyte0[])`
- `Chunk provideChunk(int i, int j)`
- `boolean chunkExists(int i, int j)`
- `void populate(IChunkProvider ichunkprovider, int i, int j)`

## ChunkProviderIso

`class implements IChunkProvider • 87 líneas`

**Proveedor de chunks optimizado para la vista isométrica del mapa.**

Carga únicamente los chunks necesarios para la proyección actual.

> **Modding:** Mejora la eficiencia al explorar mapas de gran escala.

Métodos Clave:

- `boolean chunkExists(int i, int j)`
- `Chunk provideChunk(int i, int j)`
- `void populate(IChunkProvider ichunkprovider, int i, int j)`
- `boolean saveChunks(boolean flag, IProgressUpdate iprogressupdate)`
- `boolean func_532_a()`

## ChunkProviderLoadOrGenerate

`class implements IChunkProvider • 240 líneas`

**Orquestador de carga y generación de chunks para un jugador y servidores.**

Si el chunk existe en disco, lo carga con ChunkLoader; si no, lo genera con ChunkProviderGenerate y lo guarda.

> **Modding:** Controla la persistencia de terreno y la descarga de chunks inactivos de la memoria RAM.

Métodos Clave:

- `boolean chunkExists(int i, int j)`
- `Chunk provideChunk(int i, int j)`
- `void populate(IChunkProvider ichunkprovider, int i, int j)`
- `boolean saveChunks(boolean flag, IProgressUpdate iprogressupdate)`
- `boolean func_532_a()`

## EnumSkyBlock

`enum • 43 líneas`

**Diferenciación de tipos de iluminación (Sky = Solar, Block = Bloques luminosos).**

Permite que el motor gestione dos canales de luz independientes con valores de 0 a 15 por cada bloque.

> **Modding:** Esencial en el algoritmo de cálculo de sombras e iluminación dinámica.

Métodos Clave:

- `EnumSkyBlock[] func_4151_values()`
- `EnumSkyBlock valueOf(String s)`

## Explosion

`class • 186 líneas`

**Física de detonaciones con destrucción de bloques y vectores de empuje.**

Calcula el radio de daño mediante rayos vectoriales tridimensionales, destruye bloques según su resistencia e impulsa y daña entidades.

> **Modding:** Ajusta la atenuación o añade efectos secundarios (fuego, radiación, congelación).

Métodos Clave:

- `void func_12248_a()`
- `void func_12247_b()`

## IBlockAccess

`interface • 23 líneas`

**Interfaz de solo lectura para consulta de bloques, metadatos y luz.**

Proporciona acceso a getBlockId, getBlockMetadata y getLightValue sin permitir modificaciones sobre el mundo.

> **Modding:** Permite que clases de renderizado lean el entorno de forma segura.

## IChunkLoader

`interface • 20 líneas`

**Interfaz para la persistencia de chunks en almacenamiento.**

Declara métodos de carga, guardado y descarga de datos de chunks.

> **Modding:** Implementar para crear sistemas de guardado en base de datos o formatos de región.

## IChunkProvider

`interface • 21 líneas`

**Interfaz unificada para proveedores de chunks del mundo.**

Declara métodos: loadChunk, provideChunk, populate, saveChunks y unload100OldestChunks.

> **Modding:** Implementar para crear generadores de mundos completamente personalizados.

## IWorldAccess

`interface • 29 líneas`

**Interfaz de escucha para eventos que alteran el entorno del mundo.**

Recibe notificaciones de: markBlockAndNeighborsNeedsUpdate, playSound, spawnParticle y releaseEntitySkin.

> **Modding:** Implementar para crear sistemas de monitoreo, mapas en tiempo real o grabadores de eventos.

## MetadataChunkBlock

`class • 202 líneas`

**Estructura auxiliar que empaqueta cambios de bloques en chunks para red.**

Almacena coordenadas locales y nuevos valores para transmisión masiva en paquetes de red.

> **Modding:** Parte del protocolo de sincronización multijugador.

Métodos Clave:

- `void func_4127_a(World world)`
- `boolean func_866_a(int i, int j, int k, int l, int i1, int j1)`

## NextTickListEntry

`class implements Comparable • 73 líneas`

**Entrada individual en la cola de ticks programados de World.**

Almacena coordenadas (x, y, z), ID del bloque y tiempo límite programado para su actualización.

> **Modding:** Permite programar acciones diferidas en bloques (ej. repetidores de redstone, caída de arena).

Métodos Clave:

- `boolean equals(Object obj)`
- `int hashCode()`
- `NextTickListEntry setScheduledTime(long l)`
- `int comparer(NextTickListEntry nextticklistentry)`
- `int compareTo(Object obj)`

## NibbleArray

`class • 54 líneas`

**Array de memoria comprimida que almacena 2 valores de 4 bits en cada byte.**

Reduce el consumo de memoria a la mitad para datos de metadatos (0-15) y niveles de luz de chunks (0-15).

> **Modding:** Estructura indispensable en la arquitectura de memoria de chunks de Minecraft.

Métodos Clave:

- `int getNibble(int i, int j, int k)`
- `void setNibble(int i, int j, int k, int l)`
- `boolean isValid()`

## Teleporter

`class • 302 líneas`

**Gestor cinemático de portales y translación de coordenadas entre dimensiones.**

Traduce coordenadas con escala 8:1 entre Overworld y Nether, localiza portales existentes o genera nuevos marcos de obsidiana.

> **Modding:** Modificar para implementar sistemas de teletransporte personalizados entre dimensiones.

Métodos Clave:

- `void func_4107_a(World world, Entity entity)`
- `boolean func_4106_b(World world, Entity entity)`
- `boolean func_4108_c(World world, Entity entity)`

## World

`class implements IBlockAccess • 2448 líneas`

**El gestor integral del estado del mundo, físicas, iluminación y entidades.**

La clase más extensa del juego (~2500 líneas). Mantiene listas de entidades vivas, TileEntities, chunks cargados, ciclo día/noche, clima, propagación de luz y cola de ticks.

> **Modding:** Casi cualquier mod que altere el entorno, bloques o gravedad interactúa intensamente con World.

Métodos Clave:

- `NBTTagCompound func_629_a(File file, String s)`
- `void deleteWorld(File file, String s)`
- `WorldChunkManager func_4075_a()`
- `IChunkProvider func_4081_a(File file)`
- `void func_4076_b()`

## WorldBlockPositionType

`class • 28 líneas`

**Tupla que asocia una posición tridimensional con el tipo de bloque.**

Utilizada en algoritmos de análisis ambiental.

> **Modding:** Estructura auxiliar de consulta.

## WorldChunkManager

`class • 134 líneas`

**Gestor climático y de biomas para el mundo Overworld.**

Genera capas de temperatura y humedad para clasificar cada coordenada en bosques, llanuras, desiertos, taigas o selvas.

> **Modding:** Punto de enganche para añadir nuevos biomas y variar su frecuencia o distribución geográfica.

Métodos Clave:

- `MobSpawnerBase func_4074_a(ChunkCoordIntPair chunkcoordintpair)`
- `MobSpawnerBase func_4073_a(int i, int j)`
- `double func_4072_b(int i, int j)`
- `MobSpawnerBase[] func_4069_a(int i, int j, int k, int l)`
- `double[] getTemperatures(double ad[], int i, int j, int k, int l)`

## WorldChunkManagerHell

`class extends WorldChunkManager • 66 líneas`

**Gestor de bioma único para el Inframundo (Nether).**

Devuelve condiciones homogéneas extremas de calor y aridez en todas las coordenadas.

> **Modding:** Define el clima perpetuo del Nether.

Métodos Clave:

- `MobSpawnerBase func_4074_a(ChunkCoordIntPair chunkcoordintpair)`
- `MobSpawnerBase func_4073_a(int i, int j)`
- `double func_4072_b(int i, int j)`
- `MobSpawnerBase[] func_4069_a(int i, int j, int k, int l)`
- `double[] getTemperatures(double ad[], int i, int j, int k, int l)`

## WorldClient

`class extends World • 262 líneas`

**Implementación de World especializada para el cliente multijugador.**

En lugar de simular físicas de bloques y spawn de mobs, actualiza el mundo exclusivamente a partir de los paquetes recibidos del servidor.

> **Modding:** Diferencia fundamental entre la simulación local y el cliente SMP.

Métodos Clave:

- `void tick()`
- `void func_711_c(int i, int j, int k, int l, int i1, int j1)`
- `IChunkProvider func_4081_a(File file)`
- `void func_4076_b()`
- `void func_4080_j()`

## WorldIso

`class extends World • 23 líneas`

**Mundo adaptado para la representación isométrica en el visor de mapas.**

Optimiza la consulta de bloques y colores para generar la proyección ortogonal.

> **Modding:** Ajustar para añadir capas o filtros al mapa isométrico.

Métodos Clave:

- `IChunkProvider func_4081_a(File file)`

## WorldProvider

`class • 147 líneas`

**Abstracción ambiental de dimensiones (Overworld vs Nether).**

Controla la escala de coordenadas, ciclo día/noche, niebla, color del cielo y generador de terreno aplicable.

> **Modding:** Extender para crear dimensiones completamente nuevas (ej. Aether, Luna).

Métodos Clave:

- `void registerWorld(World world)`
- `void generateLightBrightnessTable()`
- `void registerWorldChunkManager()`
- `IChunkProvider getChunkProvider()`
- `IChunkLoader getChunkLoader(File file)`

## WorldProviderHell

`class extends WorldProvider • 75 líneas`

**Implementación ambiental específica del Inframundo (Nether).**

Deshabilita el ciclo solar, fija niebla roja densa y escala las distancias en proporción 8:1.

> **Modding:** Ejemplo clave de cómo personalizar una dimensión alternativa.

Métodos Clave:

- `void registerWorldChunkManager()`
- `Vec3D func_4096_a(float f, float f1)`
- `void generateLightBrightnessTable()`
- `IChunkProvider getChunkProvider()`
- `IChunkLoader getChunkLoader(File file)`

## WorldRenderer

`class • 282 líneas`

**Compilador de fragmentos de terreno en listas de visualización OpenGL (Display Lists).**

Divide el mundo en bloques de 16x16x16 y compila su geometría estática en listas de visualización de OpenGL para que la tarjeta gráfica los dibuje a más de 60 FPS sin recalcular geometría cada fotograma.

> **Modding:** Modificar esta clase para optimizaciones de renderizado o shaders personalizados.

Métodos Clave:

- `void func_1197_a(int i, int j, int k)`
- `void func_1198_a()`
- `float func_1202_a(Entity entity)`
- `void func_1195_b()`
- `void func_1204_c()`

# 07. TileEntities y Contenedores (TileEntities & Containers) (13 clases)

Entidades asociadas a bloques específicos para almacenar datos complejos que superan los 4 bits de metadato. Incluye hornos, cofres, carteles de texto, generadores de monstruos y tocadiscos, junto con la sincronización de inventarios (Containers y Slots).

## IInventory

`interface • 23 líneas`

**Interfaz estándar para cualquier contenedor con casillas de inventario.**

Declara getSizeInventory, getStackInSlot, decrStackSize, setInventorySlotContents y getInventoryStackLimit.

**Modding:** Implementar obligatoriamente en cualquier bloque, mob o vehículo que almacene ítems.

## Slot

`class • 54 líneas`

**Representación lógica y visual de una casilla individual en un contenedor.**

Define coordenadas en pantalla (xDisplayPosition, yDisplayPosition), el inventario asociado y restricciones de inserción.

**Modding:** Extender para crear casillas con filtros de ítems (ej. solo armaduras o combustibles).

Métodos Clave:

- `void onPickupFromSlot()`
- `boolean isItemValid(ItemStack itemstack)`
- `ItemStack getStack()`
- `void putStack(ItemStack itemstack)`
- `int func_775_c()`

## SlotArmor

`class extends SlotInventory • 46 líneas`

**Casilla especializada de inventario para piezas de armadura.**

Restringe la colocación exclusivamente al tipo de armadura correspondiente (casco, peto, pantalones o botas).

**Modding:** Referencia para crear casillas con restricciones de equipamiento.

Métodos Clave:

- `int getSlotStackLimit()`
- `boolean isItemValid(ItemStack itemstack)`
- `int func_775_c()`

## SlotInventory

`class extends Slot • 30 líneas`

**Casilla de inventario estándar sin restricciones de contenido.**

Permite colocar y retirar cualquier tipo de ítem en contenedores genéricos.

**Modding:** Utilizada en cofres, hornos y almacenamiento.

Métodos Clave:

- `boolean isAtCursorPos(int i, int j)`

## TileEntity

`class • 131 líneas`

**Clase base abstracta para bloques con datos persistentes complejos y lógica por tick.**

Se vincula a una posición fija (xCoord, yCoord, zCoord). Implementa updateEntity() para ejecutarse en cada tick y serializa sus variables en etiquetas NBT.

> **Modding:** Heredar de esta clase para cualquier bloque que requiera inventario, energía o procesamiento continuo.

Métodos Clave:

- `void readFromNBT(NBTTagCompound nbttagcompound)`
- `void writeToNBT(NBTTagCompound nbttagcompound)`
- `void updateEntity()`
- `TileEntity createAndLoadEntity(NBTTagCompound nbttagcompound)`
- `int getBlockMetadata()`

## TileEntityChest

`class extends TileEntity implements IInventory • 106 líneas`

**Inventario de almacenamiento persistente para cofres (27 casillas).**

Implementa IInventory gestionando el array de ItemStacks guardados, detecta aperturas y cierres para sonidos y animación de tapa.

> **Modding:** Modificar getSizeInventory() para crear cofres con mayor capacidad.

Métodos Clave:

- `int getSizeInventory()`
- `ItemStack getStackInSlot(int i)`
- `ItemStack decrStackSize(int i, int j)`
- `void setInventorySlotContents(int i, ItemStack itemstack)`
- `String getInvName()`

## TileEntityFurnace

`class extends TileEntity implements IInventory • 293 líneas`

**Lógica interna de cocinado y quemado de combustible del horno.**

Gestiona los tiempos de cocción (furnaceCookTime), combustible restante (furnaceBurnTime), itemStacks de entrada/combustible/salida e interactúa con FurnaceRecipes.

> **Modding:** Modifica getItemBurnTime() para admitir nuevos combustibles (ej. turba, lava comprimida).

Métodos Clave:

- `int getSizeInventory()`
- `ItemStack getStackInSlot(int i)`
- `ItemStack decrStackSize(int i, int j)`
- `void setInventorySlotContents(int i, ItemStack itemstack)`
- `String getInvName()`

## TileEntityMobSpawner

`class extends TileEntity • 118 líneas`

**Generador automático de monstruos con rotación visual interna.**

Spawnea monstruos del tipo configurado en 'mobID' cuando el jugador se encuentra a menos de 16 bloques, con intervalos aleatorios de retardo.

> **Modding:** Modificar la distancia de activación o el mobID para crear spawners personalizados.

Métodos Clave:

- `boolean anyPlayerInRange()`
- `void updateEntity()`
- `void readFromNBT(NBTTagCompound nbttagcompound)`
- `void writeToNBT(NBTTagCompound nbttagcompound)`

## TileEntityMobSpawnerRenderer

`class extends TileEntitySpecialRenderer • 51 líneas`

**Renderizador 3D especial para el generador de monstruos (Mob Spawner).**

Dibuja el bloque con su jaula metálica y dibuja una miniatura flotante del mob girando continuamente en su interior.

> **Modding:** Modificar para alterar la escala o animación del mob interior.

Métodos Clave:

- `void func_931_a(TileEntityMobSpawner tileentitymobspawner, double d, double d1, double d2, float f)`
- `void renderTileEntityAt(TileEntity tileentity, double d, double d1, double d2, float f)`

## TileEntityRenderer

`class • 105 líneas`

**Despachador central de renderizado para TileEntities.**

Mapea cada clase de TileEntity con su TileEntitySpecialRenderer correspondiente e invoca el renderizado en la posición del bloque.

> **Modding:** Registrar aquí cualquier nuevo renderizador de TileEntity personalizado.

Métodos Clave:

- `TileEntitySpecialRenderer getSpecialRendererForClass(Class class1)`
- `boolean hasSpecialRenderer(TileEntity tileentity)`
- `TileEntitySpecialRenderer getSpecialRendererForEntity(TileEntity tileentity)`
- `void setRenderingContext(World world, RenderEngine renderengine, FontRenderer fontrenderer, EntityPlayer entityplayer, float f)`
- `void renderTileEntity(TileEntity tileentity, float f)`

## TileEntitySign

`class extends TileEntity • 42 líneas`

**Almacenamiento de las 4 líneas de texto del cartel.**

Guarda las cadenas 'signText[4]', gestiona su sincronización multijugador con Packet130UpdateSign y permite su renderizado 3D mediante TileEntitySignRenderer.

> **Modding:** Permite ampliar el número de líneas o añadir formato de color y comandos a carteles.

Métodos Clave:

- `void writeToNBT(NBTTagCompound nbttagcompound)`
- `void readFromNBT(NBTTagCompound nbttagcompound)`

## TileEntitySignRenderer

`class extends TileEntitySpecialRenderer • 86 líneas`

**Renderizador 3D para carteles y su texto.**

Dibuja el modelo SignModel, orienta el cartel según su rotación y renderiza las 4 líneas de texto con FontRenderer aplicando escala y centrado.

> **Modding:** Modificar para permitir texto brillante o fuentes especiales en carteles.

Métodos Clave:

- `void renderTileEntitySignAt(TileEntitySign tileentitysign, double d, double d1, double d2, float f)`
- `void renderTileEntityAt(TileEntity tileentity, double d, double d1, double d2, float f)`

## TileEntitySpecialRenderer

`class • 34 líneas`

**Clase base abstracta para renderizado 3D de TileEntities fuera de la malla estática de bloques.**

Permite emitir comandos directos de OpenGL en la posición del bloque para geometrías complejas o animadas.

> **Modding:** Heredar de esta clase para renderizar bloques dinámicos (tuberías, máquinas, cofres animados).

Métodos Clave:

- `void bindTextureByName(String s)`
- `void setTileEntityRenderer(TileEntityRenderer tileentityrenderer)`
- `FontRenderer getFontRenderer()`

# 08. Motor de Renderizado y Gráficos (Rendering Pipeline, Models & FX) (83 clases)

> La tubería gráfica basada en OpenGL 1.x/2.x y LWJGL. Destaca el Tessellator (emisión masiva de quads mediante búferes NIO nativos), listas de visualización de chunks (WorldRenderer), renderizado de bloques (RenderBlocks), modelos jerárquicos de entidades (ModelBase) y motor de partículas (EffectRenderer).

## ClippingHelper

`class • 35 líneas`

**Clase base abstracta para el cálculo del volumen de visión de la cámara (Frustum).**

Extrae los 6 planos del frustum (izquierdo, derecho, superior, inferior, cercano, lejano) de las matrices OpenGL.

> **Modding:** Base para algoritmos de descarte de geometría no visible (Frustum Culling).

Métodos Clave:

- `boolean func_1152_a(double d, double d1, double d2, double d3, double d4, double d5)`

## ClippingHelperImplementation

`class extends ClippingHelper • 99 líneas`

**Implementación concreta de ClippingHelper sobre OpenGL.**

Consulta GL_PROJECTION_MATRIX y GL_MODELVIEW_MATRIX mediante glGetFloat para actualizar los planos de corte.

> **Modding:** Permite optimizar o depurar el sistema de culling.

Métodos Clave:

- `ClippingHelper func_1155_a()`

## ColorizerFoliage

`class • 51 líneas`

**Tabla de colores 256x256 para el follaje de hojas de árboles según bioma.**

Interpola el tono verde de las hojas en función de la temperatura y humedad calculadas.

> **Modding:** Modificar foliage.png o esta clase para cambiar las paletas de color de los bosques.

Métodos Clave:

- `int func_4146_a(double d, double d1)`

## ColorizerGrass

`class • 51 líneas`

**Tabla de colores 256x256 para el césped superficial según bioma.**

Interpola el tono de verde/amarillo del bloque de césped según el clima local.

> **Modding:** Modificar grass.png o esta clase para cambiar la paleta de color del suelo.

Métodos Clave:

- `int func_4147_a(double d, double d1)`

## EffectRenderer

`class • 196 líneas`

**Gestor central del motor de partículas del juego.**

Mantiene listas de entidades EntityFX divididas en 4 capas de textura. Actualiza físicas y dibuja partículas con Tessellator.

> **Modding:** Llama a addEffect(new EntityFX(...)) para spawnear cualquier partícula personalizada en el mundo.

Métodos Clave:

- `void func_1192_a(EntityFX entityfx)`
- `void func_1193_a()`
- `void func_1189_a(Entity entity, float f)`
- `void func_1187_b(Entity entity, float f)`
- `void func_1188_a(World world)`

## FontRenderer

`class • 223 líneas`

**Renderizador de fuentes bitmap tipográficas y códigos de formato/color.**

Dibuja cadenas de texto usando el atlas 'default.png'. Procesa códigos de escape '§' (0-f) para colores de texto.

> **Modding:** Modificar para soportar fuentes de alta resolución, fuentes TrueType o códigos de formato extra.

Métodos Clave:

- `void drawStringWithShadow(String s, int i, int j, int k)`
- `void drawString(String s, int i, int j, int k)`
- `void renderString(String s, int i, int j, int k, boolean flag)`
- `int getStringWidth(String s)`

## Frustrum

`class implements ICamera • 37 líneas`

**Verificador de inclusión espacial en el cono de visión de la cámara.**

Implementa ICamera. Evalúa si una caja delimitadora AxisAlignedBB se encuentra dentro de los 6 planos de visión.

> **Modding:** Evita renderizar entidades y bloques que se encuentran a espaldas del jugador o fuera de la pantalla.

Métodos Clave:

- `void func_343_a(double d, double d1, double d2)`
- `boolean func_344_a(double d, double d1, double d2, double d3, double d4, double d5)`
- `boolean func_342_a(AxisAlignedBB axisalignedbb)`

## GLAllocation

`class • 76 líneas`

**Gestor central de asignación de memoria nativa y recursos OpenGL.**

Genera y elimina listas de visualización (glGenLists) y asigna ByteBuffers y IntBuffers en memoria directa.

> **Modding:** Utilizar sus métodos para asignar búferes nativos de forma segura sin fugas de memoria.

Métodos Clave:

- `IntBuffer createDirectIntBuffer(int i)`
- `FloatBuffer createDirectFloatBuffer(int i)`

## GuiTexturePacks

`class extends GuiScreen • 240 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiTexturePacks'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiTexturePacks'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void mouseClicked(int i, int j, int k)`
- `void mouseMovedOrUp(int i, int j, int k)`
- `void drawScreen(int i, int j, float f)`

## ICamera

`interface • 13 líneas`

**Interfaz para comprobaciones de visibilidad geométrica en la cámara.**

Declara métodos isBoundingBoxInFrustum() para validar si un objeto debe dibujarse.

> **Modding:** Permite sustituir o extender el sistema de cámara.

## ImageBuffer

`interface • 12 líneas`

**Interfaz para procesamiento de búferes de imágenes descargadas.**

Declara el método parseUserSkin para manipular los píxeles de una skin antes de subirla a la GPU.

> **Modding:** Permite implementar filtros gráficos a las texturas de los jugadores.

## ImageBufferDownload

`class implements ImageBuffer • 118 líneas`

**Procesador de skins descargadas de internet.**

Convierte la capa superior de la cabeza del skin en transparente si es necesario, replicando el formato de skins de Notch.

> **Modding:** Modificar si se implementan skins en alta resolución o con capas adicionales.

Métodos Clave:

- `BufferedImage func_883_a(BufferedImage bufferedimage)`

## IsoImageBuffer

`class • 44 líneas`

**Búfer de píxeles para almacenar fragmentos del mapa isométrico generado.**

Mantiene en memoria los bloques renderizados en la vista ortogonal.

> **Modding:** Útil para exportar mapas del mundo a imágenes de alta resolución.

Métodos Clave:

- `void func_889_a(int i, int j)`
- `void func_888_a(World world, int i, int j)`

## LoadingScreenRenderer

`class implements IProgressUpdate • 160 líneas`

**Renderizador de pantallas de carga y progreso.**

Dibuja el fondo de piedra, el texto de estado y la barra de porcentaje durante la generación de terreno o guardado.

> **Modding:** Permite personalizar la pantalla de carga.

Métodos Clave:

- `void func_596_a(String s)`
- `void func_594_b(String s)`
- `void func_597_c(String s)`
- `void displayLoadingString(String s)`
- `void setLoadingProgress(int i)`

## LogoEffectRandomizer

`class • 37 líneas`

**Efecto de perturbación y dispersión procedural para el logo de Mojang.**

Aplica una animación de desmoronamiento de píxeles al logotipo inicial.

> **Modding:** Permite crear efectos de introducción personalizados.

Métodos Clave:

- `void func_875_a()`

## ModelBase

`class • 25 líneas`

**Modelo geométrico cúbico 3D: 'ModelBase'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelBase'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelBiped

`class extends ModelBase • 153 líneas`

**Modelo geométrico cúbico 3D: 'ModelBiped'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelBiped'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelBoat

`class extends ModelBase • 52 líneas`

**Modelo geométrico cúbico 3D: 'ModelBoat'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelBoat'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelChicken

`class extends ModelBase • 75 líneas`

**Modelo geométrico cúbico 3D: 'ModelChicken'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelChicken'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelCow

`class extends ModelQuadraped • 59 líneas`

**Modelo geométrico cúbico 3D: 'ModelCow'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelCow'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelCreeper

`class extends ModelBase • 65 líneas`

**Modelo geométrico cúbico 3D: 'ModelCreeper'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelCreeper'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelGhast

`class extends ModelBase • 55 líneas`

**Modelo geométrico cúbico 3D: 'ModelGhast'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelGhast'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`
- `void render(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelMinecart

`class extends ModelBase • 57 líneas`

**Modelo geométrico cúbico 3D: 'ModelMinecart'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelMinecart'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelPig

`class extends ModelQuadraped • 19 líneas`

**Modelo geométrico cúbico 3D: 'ModelPig'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelPig'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

## ModelQuadraped

`class extends ModelBase • 60 líneas`

**Modelo geométrico cúbico 3D: 'ModelQuadraped'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelQuadraped'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelRenderer

`class • 206 líneas`

**Modelo geométrico cúbico 3D: 'ModelRenderer'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelRenderer'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void func_921_a(float f, float f1, float f2, int i, int j, int k)`
- `void addBox(float f, float f1, float f2, int i, int j, int k, float f3)`
- `void setPosition(float f, float f1, float f2)`
- `void render(float f)`
- `void func_926_b(float f)`

## ModelSheep1

`class extends ModelQuadraped • 33 líneas`

**Modelo geométrico cúbico 3D: 'ModelSheep1'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelSheep1'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

## ModelSheep2

`class extends ModelQuadraped • 20 líneas`

**Modelo geométrico cúbico 3D: 'ModelSheep2'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelSheep2'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

## ModelSkeleton

`class extends ModelZombie • 28 líneas`

**Modelo geométrico cúbico 3D: 'ModelSkeleton'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelSkeleton'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

## ModelSlime

`class extends ModelBase • 47 líneas`

**Modelo geométrico cúbico 3D: 'ModelSlime'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelSlime'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`
- `void render(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelSpider

`class extends ModelBase • 125 líneas`

**Modelo geométrico cúbico 3D: 'ModelSpider'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelSpider'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void render(float f, float f1, float f2, float f3, float f4, float f5)`
- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## ModelZombie

`class extends ModelBiped • 32 líneas`

**Modelo geométrico cúbico 3D: 'ModelZombie'.**

Define las partes corporales, cajas cúbicas y ángulos de rotación de animación para 'ModelZombie'.

> **Modding:** Modifica setRotationAngles() para alterar animaciones de movimiento o posturas.

Métodos Clave:

- `void setRotationAngles(float f, float f1, float f2, float f3, float f4, float f5)`

## OpenGlCapsChecker

`class • 23 líneas`

**Verificador de capacidades y extensiones de la tarjeta gráfica.**

Comprueba compatibilidad con OpenGL 1.2+, soporte de multitextura y capacidades de sombreado.

> **Modding:** Permite habilitar o deshabilitar efectos según la GPU del usuario.

Métodos Clave:

- `boolean checkARBOcclusion()`

## PositionTexureVertex

`class • 37 líneas`

**Vértice espacial 3D con coordenadas UV de mapeado de textura.**

Almacena una posición vectorial Vec3D y coordenadas (texturePositionX, texturePositionY) para renderizado de modelos.

> **Modding:** Utilizado en la construcción de piezas de modelos cúbicos en ModelRenderer.

Métodos Clave:

- `PositionTexureVertex setTexturePosition(float f, float f1)`

## Render

`class • 267 líneas`

**Renderizador en espacio 3D para 'Render'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void loadTexture(String s)`
- `void func_140_a(String s, String s1)`
- `void renderOffsetAABB(AxisAlignedBB axisalignedbb, double d, double d1, double d2)`
- `void renderAABB(AxisAlignedBB axisalignedbb)`
- `void setRenderManager(RenderManager rendermanager)`

## RenderArrow

`class extends Render • 79 líneas`

**Renderizador en espacio 3D para 'RenderArrow'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_154_a(EntityArrow entityarrow, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderBiped

`class extends RenderLiving • 56 líneas`

**Renderizador en espacio 3D para 'RenderBiped'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_6331_b(EntityLiving entityliving, float f)`

## RenderBlocks

`class • 2078 líneas`

**Renderizador en espacio 3D para 'RenderBlocks'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void renderBlockUsingTexture(Block block, int i, int j, int k, int l)`
- `boolean renderBlockByRenderType(Block block, int i, int j, int k)`
- `boolean renderBlockTorch(Block block, int i, int j, int k)`
- `boolean renderBlockLever(Block block, int i, int j, int k)`
- `boolean renderBlockFire(Block block, int i, int j, int k)`

## RenderBoat

`class extends Render • 50 líneas`

**Renderizador en espacio 3D para 'RenderBoat'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_157_a(EntityBoat entityboat, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderChicken

`class extends RenderLiving • 44 líneas`

**Renderizador en espacio 3D para 'RenderChicken'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_181_a(EntityChicken entitychicken, double d, double d1, double d2, float f, float f1)`
- `float func_182_a(EntityChicken entitychicken, float f)`
- `float func_170_d(EntityLiving entityliving, float f)`
- `void func_171_a(EntityLiving entityliving, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderCow

`class extends RenderLiving • 32 líneas`

**Renderizador en espacio 3D para 'RenderCow'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_177_a(EntityCow entitycow, double d, double d1, double d2, float f, float f1)`
- `void func_171_a(EntityLiving entityliving, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderCreeper

`class extends RenderLiving • 68 líneas`

**Renderizador en espacio 3D para 'RenderCreeper'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_184_a(EntityCreeper entitycreeper, float f)`
- `int func_183_a(EntityCreeper entitycreeper, float f, float f1)`
- `void func_6330_a(EntityLiving entityliving, float f)`
- `int func_173_a(EntityLiving entityliving, float f, float f1)`

## RenderEngine

`class • 469 líneas`

**Renderizador en espacio 3D para 'RenderEngine'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `int getTexture(String s)`
- `int allocateAndSetupTexture(BufferedImage bufferedimage)`
- `void setupTexture(BufferedImage bufferedimage, int i)`
- `void deleteTexture(int i)`
- `int getTextureForDownloadableImage(String s, String s1)`

## RenderEntity

`class extends Render • 22 líneas`

**Renderizador en espacio 3D para 'RenderEntity'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderFallingSand

`class extends Render • 38 líneas`

**Renderizador en espacio 3D para 'RenderFallingSand'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_156_a(EntityFallingSand entityfallingsand, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderFireball

`class extends Render • 51 líneas`

**Renderizador en espacio 3D para 'RenderFireball'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_4012_a(EntityFireball entityfireball, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderFish

`class extends Render • 92 líneas`

**Renderizador en espacio 3D para 'RenderFish'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_4011_a(EntityFish entityfish, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderGhast

`class extends RenderLiving • 35 líneas`

**Renderizador en espacio 3D para 'RenderGhast'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_4014_a(EntityGhast entityghast, float f)`
- `void func_6330_a(EntityLiving entityliving, float f)`

## RenderGlobal

`class implements IWorldAccess • 1286 líneas`

**Renderizador en espacio 3D para 'RenderGlobal'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_946_a(World world)`
- `void func_958_a()`
- `void func_951_a(Vec3D vec3d, ICamera icamera, float f)`
- `String func_953_b()`
- `String func_957_c()`

## RenderHelper

`class • 63 líneas`

**Renderizador en espacio 3D para 'RenderHelper'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void disableStandardItemLighting()`
- `void enableStandardItemLighting()`

## RenderItem

`class extends Render • 218 líneas`

**Renderizador en espacio 3D para 'RenderItem'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_165_a(EntityItem entityitem, double d, double d1, double d2, float f, float f1)`
- `void renderItemIntoGUI(FontRenderer fontrenderer, RenderEngine renderengine, ItemStack itemstack, int i, int j)`
- `void renderItemOverlayIntoGUI(FontRenderer fontrenderer, RenderEngine renderengine, ItemStack itemstack, int i, int j)`
- `void renderTexturedQuad(int i, int j, int k, int l, int i1, int j1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderList

`class • 87 líneas`

**Renderizador en espacio 3D para 'RenderList'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_861_a(int i, int j, int k, double d, double d1, double d2)`
- `boolean func_862_a(int i, int j, int k)`
- `void func_858_a(int i)`
- `void func_860_a()`
- `void func_859_b()`

## RenderLiving

`class extends Render • 173 líneas`

**Renderizador en espacio 3D para 'RenderLiving'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_4013_a(ModelBase modelbase)`
- `void func_171_a(EntityLiving entityliving, double d, double d1, double d2, float f, float f1)`
- `float func_167_c(EntityLiving entityliving, float f)`
- `float func_170_d(EntityLiving entityliving, float f)`
- `void func_6331_b(EntityLiving entityliving, float f)`

## RenderManager

`class • 134 líneas`

**Renderizador en espacio 3D para 'RenderManager'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `Render func_4117_a(Class class1)`
- `Render func_855_a(Entity entity)`
- `void func_857_a(World world, RenderEngine renderengine, FontRenderer fontrenderer, EntityPlayer entityplayer, GameSettings gamesettings, float f)`
- `void func_854_a(Entity entity, float f)`
- `void func_853_a(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderMinecart

`class extends Render • 95 líneas`

**Renderizador en espacio 3D para 'RenderMinecart'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_152_a(EntityMinecart entityminecart, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderPainting

`class extends Render • 136 líneas`

**Renderizador en espacio 3D para 'RenderPainting'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_158_a(EntityPainting entitypainting, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderPig

`class extends RenderLiving • 26 líneas`

**Renderizador en espacio 3D para 'RenderPig'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `boolean func_180_a(EntityPig entitypig, int i)`
- `boolean func_166_a(EntityLiving entityliving, int i)`

## RenderPlayer

`class extends RenderLiving • 235 líneas`

**Renderizador en espacio 3D para 'RenderPlayer'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `boolean func_187_a(EntityPlayer entityplayer, int i)`
- `void func_188_a(EntityPlayer entityplayer, double d, double d1, double d2, float f, float f1)`
- `void func_4015_a(EntityPlayer entityplayer, float f)`
- `void func_186_b(EntityPlayer entityplayer, float f)`
- `void drawFirstPersonHand()`

## RenderSheep

`class extends RenderLiving • 26 líneas`

**Renderizador en espacio 3D para 'RenderSheep'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `boolean func_176_a(EntitySheep entitysheep, int i)`
- `boolean func_166_a(EntityLiving entityliving, int i)`

## RenderSlime

`class extends RenderLiving • 54 líneas`

**Renderizador en espacio 3D para 'RenderSlime'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `boolean func_179_a(EntitySlime entityslime, int i)`
- `void func_178_a(EntitySlime entityslime, float f)`
- `void func_6330_a(EntityLiving entityliving, float f)`
- `boolean func_166_a(EntityLiving entityliving, int i)`

## RenderSnowball

`class extends Render • 50 líneas`

**Renderizador en espacio 3D para 'RenderSnowball'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_155_a(EntitySnowball entitysnowball, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderSorter

`class implements Comparator • 50 líneas`

**Renderizador en espacio 3D para 'RenderSorter'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `int func_993_a(WorldRenderer worldrenderer, WorldRenderer worldrenderer1)`
- `int compare(Object obj, Object obj1)`

## RenderSpider

`class extends RenderLiving • 52 líneas`

**Renderizador en espacio 3D para 'RenderSpider'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `float func_191_a(EntitySpider entityspider)`
- `boolean func_190_a(EntitySpider entityspider, int i)`
- `float func_172_a(EntityLiving entityliving)`
- `boolean func_166_a(EntityLiving entityliving, int i)`

## RenderTNTPrimed

`class extends Render • 64 líneas`

**Renderizador en espacio 3D para 'RenderTNTPrimed'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_153_a(EntityTNTPrimed entitytntprimed, double d, double d1, double d2, float f, float f1)`
- `void doRender(Entity entity, double d, double d1, double d2, float f, float f1)`

## RenderZombieSimple

`class extends RenderLiving • 28 líneas`

**Renderizador en espacio 3D para 'RenderZombieSimple'.**

Dibuja la entidad o elemento en pantalla conectando su textura con el modelo 3D correspondiente.

> **Modding:** Sobrescribe doRender() para añadir efectos visuales o cambiar la textura.

Métodos Clave:

- `void func_175_a(EntityZombieSimple entityzombiesimple, float f)`
- `void func_6330_a(EntityLiving entityliving, float f)`

## ScaledResolution

`class • 32 líneas`

**Calculador dinámico del factor de escala de la interfaz de usuario (GUI Scale).**

Calcula la resolución lógica (scaledWidth, scaledHeight) y el factor de escala según el tamaño de la ventana.

> **Modding:** Imprescindible para que los menús y HUD se adapten correctamente a cualquier resolución de pantalla.

Métodos Clave:

- `int getScaledWidth()`
- `int getScaledHeight()`

## SignModel

`class • 26 líneas`

**Modelo geométrico 3D del cartel (tablero y poste de soporte).**

Renderizado por TileEntitySignRenderer para dar volumen al cartel en el espacio.

> **Modding:** Permite alterar la forma física del cartel.

Métodos Clave:

- `void func_887_a()`

## TerrainTextureManager

`class • 268 líneas`

**Administrador de atlas y texturas del terreno.**

Supervisa la asignación y carga de mosaicos en terrain.png.

> **Modding:** Utilizado en la configuración del atlas de bloques.

Métodos Clave:

- `void func_799_a(IsoImageBuffer isoimagebuffer)`

## Tessellator

`class • 355 líneas`

**Motor de emisión geométrica de quads de alto rendimiento con búferes nativos.**

La clase gráfica más crítica del cliente. Acumula vértices, normales, UVs y colores en un ByteBuffer nativo, volcando miles de polígonos a la GPU con una sola llamada glDrawArrays.

> **Modding:** Cualquier dibujo de geometría personalizada en el mundo debe utilizar Tessellator.instance para máximo rendimiento.

Métodos Clave:

- `void draw()`
- `void startDrawingQuads()`
- `void startDrawing(int i)`
- `void setTextureUV(double d, double d1)`
- `void setColorOpaque_F(float f, float f1, float f2)`

## TextureCompassFX

`class extends TextureFX • 137 líneas`

**Animación procedural para la esfera de la brújula en el inventario.**

Calcula el ángulo hacia el punto de spawn del mundo respecto a la orientación del jugador y rota la aguja suavemente con inercia.

> **Modding:** Modificar para que la brújula apunte a coordenadas personalizadas o bases de jugadores.

Métodos Clave:

- `void func_783_a()`

## TextureFX

`class • 43 líneas`

**Clase base para animaciones procedimentales de textura en memoria.**

Almacena un búfer de píxeles (imageData) que se actualiza en cada tick antes de enviarse a OpenGL mediante glTexSubImage2D.

> **Modding:** Heredar de esta clase para crear texturas animadas personalizadas (fluidos, energía, pantallas).

Métodos Clave:

- `void func_783_a()`
- `void func_782_a(RenderEngine renderengine)`

## TextureFlamesFX

`class extends TextureFX • 92 líneas`

**Animación procedimental para las partículas y texturas de fuego.**

Simula llamas ascendentes con degradados térmicos dinámicos.

> **Modding:** Modificar para crear fuego azul, verde o mágico.

Métodos Clave:

- `void func_783_a()`

## TextureLavaFX

`class extends TextureFX • 93 líneas`

**Animación procedimental para el bloque de lava estacionaria.**

Simula burbujeo y corrientes incandescentes con transiciones de color naranja y rojo.

> **Modding:** Modificar para personalizar la apariencia de fluidos ígneos.

Métodos Clave:

- `void func_783_a()`

## TextureLavaFlowFX

`class extends TextureFX • 97 líneas`

**Animación procedimental para la lava en flujo continuo.**

Calcula el desplazamiento visual de la corriente de lava en movimiento horizontal y vertical.

> **Modding:** Complementa TextureLavaFX.

Métodos Clave:

- `void func_783_a()`

## TexturePackBase

`class • 45 líneas`

**Clase base abstracta para paquetes de texturas (Texture Packs).**

Declara métodos para abrir streams de recursos gráficos, leer la descripción del paquete y el icono pack.png.

> **Modding:** Base del sistema de personalización gráfica del juego.

Métodos Clave:

- `void func_6482_a()`
- `void closeTexturePackFile()`
- `void func_6485_a(Minecraft minecraft)`
- `void func_6484_b(Minecraft minecraft)`
- `void func_6483_c(Minecraft minecraft)`

## TexturePackCustom

`class extends TexturePackBase • 138 líneas`

**Paquete de texturas personalizado cargado desde un archivo .zip en la carpeta texturepacks.**

Extrae imágenes, sonidos y fuentes desde el archivo comprimido sin descomprimirlo en disco.

> **Modding:** Permite a los usuarios sustituir los gráficos del juego.

Métodos Clave:

- `void func_6485_a(Minecraft minecraft)`
- `void func_6484_b(Minecraft minecraft)`
- `void func_6483_c(Minecraft minecraft)`
- `void func_6482_a()`
- `void closeTexturePackFile()`

## TexturePackDefault

`class extends TexturePackBase • 55 líneas`

**Paquete de texturas por defecto integrado en el cliente.**

Carga los recursos originales directamente desde el classpath del juego.

> **Modding:** Paquete de respaldo permanente ante archivos faltantes.

Métodos Clave:

- `void func_6484_b(Minecraft minecraft)`
- `void func_6483_c(Minecraft minecraft)`

## TexturePackList

`class • 115 líneas`

**Gestor del catálogo de paquetes de texturas disponibles en disco.**

Escanea la carpeta texturepacks, detecta archivos ZIP añadidos por el usuario y permite alternar el paquete activo.

> **Modding:** Modificar si se requiere forzar un paquete de recursos en un modpack.

Métodos Clave:

- `boolean setTexturePack(TexturePackBase texturepackbase)`
- `void func_6532_a()`
- `List availableTexturePacks()`

## TexturePortalFX

`class extends TextureFX • 100 líneas`

**Animación procedimental para la lámina del portal al Inframundo.**

Genera remolinos de ruido púrpura hipnótico dentro del portal de obsidiana.

> **Modding:** Personalizar para crear portales con efectos visuales únicos.

Métodos Clave:

- `void func_783_a()`

## TextureWatchFX

`class extends TextureFX • 105 líneas`

**Animación procedural para la esfera del reloj en el inventario.**

Calcula la posición del sol y la luna a partir de World.worldTime y rota el dial celeste en tiempo real.

> **Modding:** Referencia para crear diales temporales o calendarios en ítems.

Métodos Clave:

- `void func_783_a()`

## TextureWaterFX

`class extends TextureFX • 99 líneas`

**Animación procedimental para el bloque de agua estacionaria.**

Simula ondas y perturbaciones continuas en la superficie del agua calculando fluctuaciones físicas de píxeles.

> **Modding:** Modificar para cambiar el estilo o color del agua animada.

Métodos Clave:

- `void func_783_a()`

## TexturedQuad

`class • 65 líneas`

**Polígono cuadrilátero formado por 4 vértices PositionTexureVertex.**

Dibuja una cara plana con textura mapeada en el espacio tridimensional calculando el vector normal para iluminación.

> **Modding:** El elemento constructivo básico de las cajas en ModelRenderer.

Métodos Clave:

- `void func_809_a()`
- `void func_808_a(Tessellator tessellator, float f)`

## TexureWaterFlowFX

`class extends TextureFX • 100 líneas`

**Animación procedimental para el flujo horizontal del agua en movimiento.**

Modifica píxel a píxel el búfer de imagen para simular olas y corrientes de agua animadas.

> **Modding:** Permite personalizar la apariencia de fluidos en movimiento.

Métodos Clave:

- `void func_783_a()`

# 09. Interfaz de Usuario y Menús (GUI, HUD & Screens) (30 clases)

El sistema de interfaz 2D. Maneja el HUD del juego (vida, oxígeno, hotbar, mira), menús principales, menús de inventario, cofres y hornos, pantallas de opciones y controles, pantalla de muerte, selección de mundos y chat.

## ChatLine

`class • 18 líneas`

**Representación de una línea individual de texto en el historial del chat.**

Almacena el texto del mensaje y un contador de tiempo de vida (updateCounter) para su desvanecimiento gradual.

**Modding:** Permite modificar la duración de visibilidad de los mensajes o añadir marcas de tiempo (timestamps).

## Gui

`class • 91 líneas`

**Pantalla o elemento de interfaz gráfica: 'Gui'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'Gui'.

**Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void drawRect(int i, int j, int k, int l, int i1)`
- `void drawGradientRect(int i, int j, int k, int l, int i1, int j1)`
- `void drawCenteredString(FontRenderer fontrenderer, String s, int i, int j, int k)`
- `void drawString(FontRenderer fontrenderer, String s, int i, int j, int k)`
- `void drawTexturedModalRect(int i, int j, int k, int l, int i1, int j1)`

## GuiButton

`class extends Gui • 93 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiButton'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiButton'.

**Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `int getHoverState(boolean flag)`
- `void drawButton(Minecraft minecraft, int i, int j)`
- `void mouseDragged(Minecraft minecraft, int i, int j)`
- `void mouseReleased(int i, int j)`
- `boolean mousePressed(Minecraft minecraft, int i, int j)`

## GuiChat

`class extends GuiScreen • 87 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiChat'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiChat'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void onGuiClosed()`
- `void updateScreen()`
- `void keyTyped(char c, int i)`
- `void drawScreen(int i, int j, float f)`

## GuiChest

`class extends GuiContainer • 69 líneas`

**Pantalla del inventario de cofre simple (27 slots) o doble (54 slots).**

Muestra las casillas del cofre en la mitad superior y el inventario del jugador en la mitad inferior para transferencia de ítems.

> **Modding:** Permite añadir botones de ordenación automática de cofres.

Métodos Clave:

- `void drawGuiContainerForegroundLayer()`
- `void drawGuiContainerBackgroundLayer(float f)`

## GuiConflictWarning

`class extends GuiScreen • 52 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiConflictWarning'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiConflictWarning'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void updateScreen()`
- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## GuiConnectFailed

`class extends GuiScreen • 50 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiConnectFailed'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiConnectFailed'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void updateScreen()`
- `void keyTyped(char c, int i)`
- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## GuiConnecting

`class extends GuiScreen • 82 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiConnecting'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiConnecting'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void updateScreen()`
- `void keyTyped(char c, int i)`
- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## GuiContainer

`class extends GuiScreen • 278 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiContainer'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiContainer'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void drawScreen(int i, int j, float f)`
- `void drawGuiContainerForegroundLayer()`
- `void mouseClicked(int i, int j, int k)`
- `void mouseMovedOrUp(int i, int j, int k)`
- `void keyTyped(char c, int i)`

## GuiControls

`class extends GuiScreen • 71 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiControls'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiControls'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void keyTyped(char c, int i)`
- `void drawScreen(int i, int j, float f)`

## GuiDeleteWorld

`class extends GuiSelectWorld • 41 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiDeleteWorld'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiDeleteWorld'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui2()`
- `void selectWorld(int i)`
- `void deleteWorld(boolean flag, int i)`

## GuiDownloadTerrain

`class extends GuiScreen • 52 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiDownloadTerrain'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiDownloadTerrain'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void keyTyped(char c, int i)`
- `void initGui()`
- `void updateScreen()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## GuiEditSign

`class extends GuiScreen • 119 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiEditSign'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiEditSign'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void onGuiClosed()`
- `void updateScreen()`
- `void actionPerformed(GuiButton guibutton)`
- `void keyTyped(char c, int i)`

## GuiFurnace

`class extends GuiContainer • 59 líneas`

**Pantalla interactiva del horno de fundición.**

Dibuja la animación de la llama de fuego consumiéndose y la flecha de progreso del cocinado del material.

> **Modding:** Modificar si se añaden ranuras adicionales de combustible o mejoras (upgrades).

Métodos Clave:

- `void drawGuiContainerForegroundLayer()`
- `void drawGuiContainerBackgroundLayer(float f)`

## GuiGameOver

`class extends GuiScreen • 62 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiGameOver'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiGameOver'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void keyTyped(char c, int i)`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`
- `boolean doesGuiPauseGame()`

## GuiIngame

`class extends Gui • 394 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiIngame'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiIngame'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void renderGameOverlay(float f, boolean flag, int i, int j)`
- `void func_555_a()`
- `void addChatMessage(String s)`
- `void func_553_b(String s)`

## GuiIngameMenu

`class extends GuiScreen • 76 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiIngameMenu'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiIngameMenu'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void updateScreen()`
- `void drawScreen(int i, int j, float f)`

## GuiInventory

`class extends GuiContainer • 101 líneas`

**Pantalla interactiva del inventario personal del jugador (tecla E).**

Muestra la vista previa 3D del modelo del jugador girando, las 4 casillas de armadura, la cuadrícula de crafteo 2x2, la casilla de resultado y el inventario general.

> **Modding:** Permite añadir slots especiales de accesorios (anillos, capas) o botones adicionales.

Métodos Clave:

- `void drawGuiContainerForegroundLayer()`
- `void drawScreen(int i, int j, float f)`
- `void drawGuiContainerBackgroundLayer(float f)`

## GuiMainMenu

`class extends GuiScreen • 251 líneas`

**Pantalla del menú principal del juego.**

Renderiza el logotipo tridimensional de Minecraft, los textos amarillos oscilantes ('Splash text') y los botones de 'Singleplayer', 'Multiplayer', 'Mods and Texture Packs' y 'Options'.

> **Modding:** Modifica los botones o el fondo para personalizar la pantalla de bienvenida de un modpack.

Métodos Clave:

- `void updateScreen()`
- `void keyTyped(char c, int i)`
- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## GuiMultiplayer

`class extends GuiScreen • 120 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiMultiplayer'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiMultiplayer'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void updateScreen()`
- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void keyTyped(char c, int i)`
- `void drawScreen(int i, int j, float f)`

## GuiOptions

`class extends GuiScreen • 70 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiOptions'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiOptions'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## GuiScreen

`class extends Gui • 196 líneas`

**Clase base para todas las pantallas interactivas de menús y diálogos.**

Maneja el ciclo de vida de una pantalla: initGui() (configuración de botones), actionPerformed() (manejo de clics), keyTyped() (teclado), y drawScreen() (dibujo de elementos 2D).

> **Modding:** Cualquier nuevo menú o panel interactivo debe extender GuiScreen.

Métodos Clave:

- `void drawScreen(int i, int j, float f)`
- `void keyTyped(char c, int i)`
- `String getClipboardString()`
- `void mouseClicked(int i, int j, int k)`
- `void mouseMovedOrUp(int i, int j, int k)`

## GuiSelectWorld

`class extends GuiScreen • 98 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiSelectWorld'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiSelectWorld'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `String getWorldName(int i)`
- `void initGui2()`
- `void actionPerformed(GuiButton guibutton)`
- `void selectWorld(int i)`

## GuiSlider

`class extends GuiButton • 83 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiSlider'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiSlider'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `int getHoverState(boolean flag)`
- `void mouseDragged(Minecraft minecraft, int i, int j)`
- `boolean mousePressed(Minecraft minecraft, int i, int j)`
- `void mouseReleased(int i, int j)`

## GuiSmallButton

`class extends GuiButton • 14 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiSmallButton'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiSmallButton'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

## GuiUnused

`class extends GuiScreen • 28 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiUnused'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiUnused'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void drawScreen(int i, int j, float f)`
- `void keyTyped(char c, int i)`

## GuiYesNo

`class extends GuiScreen • 42 líneas`

**Pantalla o elemento de interfaz gráfica: 'GuiYesNo'.**

Controla los botones, textos, fondos y eventos de ratón/teclado para la interfaz 'GuiYesNo'.

> **Modding:** Sobrescribe initGui, actionPerformed o drawScreen para personalizar la interfaz.

Métodos Clave:

- `void initGui()`
- `void actionPerformed(GuiButton guibutton)`
- `void drawScreen(int i, int j, float f)`

## IProgressUpdate

`interface • 15 líneas`

**Interfaz para receptores de progreso de tareas largas.**

Permite que World actualice el texto y porcentaje en LoadingScreenRenderer durante la carga o guardado.

> **Modding:** Usar para mostrar barras de progreso en procesos extensos de mods.

## Packet3Chat

`class extends Packet • 41 líneas`

**Paquete de datos de red: 'Packet3Chat' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet3Chat'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## ScreenShotHelper

`class • 75 líneas`

**Utilidad para capturar pantallas de juego y guardarlas en disco.**

Lee el framebuffer mediante glReadPixels, convierte el formato de color BGR a RGB y escribe un archivo PNG en la carpeta screenshots.

> **Modding:** Modificar si se desea añadir marcas de agua o formatos de compresión alternativos a las capturas.

Métodos Clave:

- `String func_4148_a(File file, int i, int j)`

# 10. Redes y Protocolo Multijugador (Networking & Packets) (46 clases)

> La arquitectura cliente-servidor de Alpha 1.2.6 (SMP). Protocolo basado en sockets TCP con hilos asíncronos de lectura y escritura (NetworkManager), y el catálogo completo de paquetes binarios (0x00 a 0xFF) para sincronización de entidades, bloques, inventarios y chat.

## NetClientHandler

`class extends NetHandler • 557 líneas`

**Despachador central de paquetes de red en el cliente multijugador.**

Recibe los paquetes del servidor y los traduce en modificaciones concretas del mundo local: creación de entidades, bloques, salud y chat.

> **Modding:** Punto clave para interceptar o añadir nuevos tipos de mensajes entre cliente y servidor.

Métodos Clave:

- `void processReadPackets()`
- `void handleLogin(Packet1Login packet1login)`
- `void handlePickupSpawn(Packet21PickupSpawn packet21pickupspawn)`
- `void handleVehicleSpawn(Packet23VehicleSpawn packet23vehiclespawn)`
- `void func_6498_a(Packet28 packet28)`

## NetHandler

`class • 185 líneas`

**Manejador base de eventos del protocolo de paquetes de red.**

Declara métodos vacíos para todos los paquetes (handleLogin, handleChat, handleBlockDig, etc.).

> **Modding:** Base abstracta que implementan NetClientHandler y NetServerHandler.

Métodos Clave:

- `void handleMapChunk(Packet51MapChunk packet51mapchunk)`
- `void func_4114_b(Packet packet)`
- `void handleErrorMessage(String s)`
- `void handleKickDisconnect(Packet255KickDisconnect packet255kickdisconnect)`
- `void handleLogin(Packet1Login packet1login)`

## NetworkManager

`class • 239 líneas`

**Gestor multihilo de sockets TCP para comunicaciones de red SMP.**

Mantiene hilos dedicados para lectura y escritura asíncrona sobre sockets TCP, administrando colas de paquetes entrantes y salientes.

> **Modding:** Modificar esta clase para cifrado de red, compresión de paquetes o proxies.

Métodos Clave:

- `void addToSendQueue(Packet packet)`
- `void networkShutdown(String s)`
- `void processReadPackets()`

## NetworkMasterThread

`class extends Thread • 44 líneas`

**Hilo maestro supervisor de la conexión de red TCP.**

Supervisa el estado de salud de los hilos de lectura y escritura del socket.

> **Modding:** Garantiza el cierre limpio de recursos en caso de caída de conexión.

Métodos Clave:

- `void run()`

## NetworkReaderThread

`class extends Thread • 36 líneas`

**Hilo secundario para la recepción continua de paquetes de red desde el socket.**

Lee bytes del DataInputStream, deserializa el paquete y lo encola en la lista de paquetes entrantes.

> **Modding:** Evita que la latencia de la red bloquee el hilo principal de renderizado.

Métodos Clave:

- `void run()`

## NetworkWriterThread

`class extends Thread • 36 líneas`

**Hilo secundario para la transmisión asíncrona de paquetes hacia el socket TCP.**

Toma paquetes de la cola de salida, los serializa en DataOutputStream y los vacía en el flujo de red.

> **Modding:** Garantiza una emisión fluida de paquetes sin retrasar los ticks del juego.

Métodos Clave:

- `void run()`

## Packet

`class • 152 líneas`

**Paquete de datos de red: 'Packet' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `Packet getNewPacket(int i)`
- `int getPacketId()`
- `Packet readPacket(DataInputStream datainputstream)`
- `void writePacket(Packet packet, DataOutputStream dataoutputstream)`

## Packet0KeepAlive

`class extends Packet • 31 líneas`

**Paquete de datos de red: 'Packet0KeepAlive' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet0KeepAlive'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void processPacket(NetHandler nethandler)`
- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet10Flying

`class extends Packet • 49 líneas`

**Paquete de datos de red: 'Packet10Flying' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet10Flying'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void processPacket(NetHandler nethandler)`
- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet11PlayerPosition

`class extends Packet10Flying • 48 líneas`

**Paquete de datos de red: 'Packet11PlayerPosition' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet11PlayerPosition'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet12PlayerLook

`class extends Packet10Flying • 42 líneas`

**Paquete de datos de red: 'Packet12PlayerLook' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet12PlayerLook'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet13PlayerLookMove

`class extends Packet10Flying • 56 líneas`

**Paquete de datos de red: 'Packet13PlayerLookMove' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet13PlayerLookMove'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet14BlockDig

`class extends Packet • 57 líneas`

**Paquete de datos de red: 'Packet14BlockDig' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet14BlockDig'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet15Place

`class extends Packet • 57 líneas`

**Paquete de datos de red: 'Packet15Place' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet15Place'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet16BlockItemSwitch

`class extends Packet • 45 líneas`

**Paquete de datos de red: 'Packet16BlockItemSwitch' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet16BlockItemSwitch'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet17AddToInventory

`class extends Packet • 42 líneas`

**Paquete de datos de red: 'Packet17AddToInventory' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet17AddToInventory'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet18ArmAnimation

`class extends Packet • 45 líneas`

**Paquete de datos de red: 'Packet18ArmAnimation' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet18ArmAnimation'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet1Login

`class extends Packet • 55 líneas`

**Paquete de datos de red: 'Packet1Login' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet1Login'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet20NamedEntitySpawn

`class extends Packet • 70 líneas`

**Paquete de datos de red: 'Packet20NamedEntitySpawn' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet20NamedEntitySpawn'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet21PickupSpawn

`class extends Packet • 73 líneas`

**Paquete de datos de red: 'Packet21PickupSpawn' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet21PickupSpawn'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet22Collect

`class extends Packet • 39 líneas`

**Paquete de datos de red: 'Packet22Collect' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet22Collect'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet23VehicleSpawn

`class extends Packet • 48 líneas`

**Paquete de datos de red: 'Packet23VehicleSpawn' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet23VehicleSpawn'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet24MobSpawn

`class extends Packet • 65 líneas`

**Paquete de datos de red: 'Packet24MobSpawn' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet24MobSpawn'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet255KickDisconnect

`class extends Packet • 41 líneas`

**Paquete de datos de red: 'Packet255KickDisconnect' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet255KickDisconnect'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet28

`class extends Packet • 83 líneas`

**Paquete de datos de red: 'Packet28' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet28'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet29DestroyEntity

`class extends Packet • 36 líneas`

**Paquete de datos de red: 'Packet29DestroyEntity' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet29DestroyEntity'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet2Handshake

`class extends Packet • 41 líneas`

**Paquete de datos de red: 'Packet2Handshake' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet2Handshake'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet30Entity

`class extends Packet • 43 líneas`

**Paquete de datos de red: 'Packet30Entity' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet30Entity'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet31RelEntityMove

`class extends Packet30Entity • 35 líneas`

**Paquete de datos de red: 'Packet31RelEntityMove' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet31RelEntityMove'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet32EntityLook

`class extends Packet30Entity • 34 líneas`

**Paquete de datos de red: 'Packet32EntityLook' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet32EntityLook'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet33RelEntityMoveLook

`class extends Packet30Entity • 40 líneas`

**Paquete de datos de red: 'Packet33RelEntityMoveLook' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet33RelEntityMoveLook'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## Packet34EntityTeleport

`class extends Packet • 61 líneas`

**Paquete de datos de red: 'Packet34EntityTeleport' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet34EntityTeleport'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet38

`class extends Packet • 39 líneas`

**Paquete de datos de red: 'Packet38' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet38'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet39

`class extends Packet • 39 líneas`

**Paquete de datos de red: 'Packet39' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet39'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `int getPacketSize()`
- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`

## Packet4UpdateTime

`class extends Packet • 36 líneas`

**Paquete de datos de red: 'Packet4UpdateTime' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet4UpdateTime'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet50PreChunk

`class extends Packet • 42 líneas`

**Paquete de datos de red: 'Packet50PreChunk' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet50PreChunk'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet51MapChunk

`class extends Packet • 76 líneas`

**Paquete de datos de red: 'Packet51MapChunk' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet51MapChunk'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet52MultiBlockChange

`class extends Packet • 63 líneas`

**Paquete de datos de red: 'Packet52MultiBlockChange' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet52MultiBlockChange'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet53BlockChange

`class extends Packet • 49 líneas`

**Paquete de datos de red: 'Packet53BlockChange' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet53BlockChange'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet59ComplexEntity

`class extends Packet • 69 líneas`

**Paquete de datos de red: 'Packet59ComplexEntity' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet59ComplexEntity'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet5PlayerInventory

`class extends Packet • 75 líneas`

**Paquete de datos de red: 'Packet5PlayerInventory' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet5PlayerInventory'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet60

`class extends Packet • 75 líneas`

**Paquete de datos de red: 'Packet60' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet60'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet6SpawnPosition

`class extends Packet • 42 líneas`

**Paquete de datos de red: 'Packet6SpawnPosition' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet6SpawnPosition'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet7

`class extends Packet • 49 líneas`

**Paquete de datos de red: 'Packet7' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet7'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet8

`class extends Packet • 36 líneas`

**Paquete de datos de red: 'Packet8' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet8'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `void processPacket(NetHandler nethandler)`
- `int getPacketSize()`

## Packet9

`class extends Packet • 32 líneas`

**Paquete de datos de red: 'Packet9' en el protocolo SMP.**

Serializa y deserializa información binaria a través de sockets TCP para la acción 'Packet9'.

> **Modding:** Añade nuevos campos o sincronizaciones modificando readPacketData y writePacketData.

Métodos Clave:

- `void processPacket(NetHandler nethandler)`
- `void readPacketData(DataInputStream datainputstream)`
- `void writePacketData(DataOutputStream dataoutputstream)`
- `int getPacketSize()`

## 11. Sistema NBT y Serialización (Named Binary Tag) (13 clases)

El formato de archivo binario jerárquico ideado por Notch. Estructuras de datos tipadas en árbol (Byte, Short, Int, Long, Float, Double, ByteArray, String, List, Compound) con soporte de compresión y descompresión GZIP para guardar jugadores, mundos y chunks.

### CompressedStreamTools

`class • 89 líneas`

**Utilidades estáticas para lectura y escritura de estructuras NBT comprimidas con GZIP.**

Serializa árboles NBT hacia flujos de datos o archivos en disco con compresión transparente.

**Modding:** Usar para guardar y cargar configuraciones personalizadas o estructuras en disco.

Métodos Clave:

- `NBTTagCompound func_1138_a(InputStream inputstream)`
- `void writeGzippedCompoundToOutputStream(NBTTagCompound nbttagcompound, OutputStream outputstream)`
- `NBTTagCompound func_1140_a(byte abyte0[])`
- `byte[] func_1142_a(NBTTagCompound nbttagcompound)`
- `NBTTagCompound func_1141_a(DataInput datainput)`

### NBTBase

`class • 149 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTBase'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTBase'.

**Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `String getKey()`
- `NBTBase setKey(String s)`
- `NBTBase readTag(DataInput datainput)`
- `void writeTag(NBTBase nbtbase, DataOutput dataoutput)`
- `NBTBase createTagOfType(byte byte0)`

### NBTTagByte

`class extends NBTBase • 41 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagByte'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagByte'.

**Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagByteArray

`class extends NBTBase • 44 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagByteArray'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagByteArray'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagCompound

`class extends NBTBase • 221 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagCompound'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagCompound'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `void setTag(String s, NBTBase nbtbase)`
- `void setByte(String s, byte byte0)`
- `void setShort(String s, short word0)`
- `void setInteger(String s, int i)`

## NBTTagDouble

`class extends NBTBase • 41 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagDouble'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagDouble'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagEnd

`class extends NBTBase • 32 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagEnd'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagEnd'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagFloat

`class extends NBTBase • 41 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagFloat'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagFloat'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagInt

`class extends NBTBase • 41 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagInt'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagInt'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagList

`class extends NBTBase • 78 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagList'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagList'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`
- `void setTag(NBTBase nbtbase)`
- `NBTBase tagAt(int i)`
- `int tagCount()`

## NBTTagLong

`class extends NBTBase • 41 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagLong'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagLong'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagShort

`class extends NBTBase • 41 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagShort'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagShort'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## NBTTagString

`class extends NBTBase • 48 líneas`

**Nodo de almacenamiento serializable NBT: 'NBTTagString'.**

Almacena datos en el formato binario jerárquico Named Binary Tag para 'NBTTagString'.

> **Modding:** Utilizado para guardar variables personalizadas en mundos, jugadores y entidades.

Métodos Clave:

- `byte getType()`
- `String toString()`

## 12. Motor de Audio y Sonido (Sound Engine & Music) (5 clases)

> Integración con la librería Paulscode SoundSystem sobre OpenAL. Carga y reproduce efectos de sonido aleatorizados en tono/volumen (SoundPool), decodifica audio Ogg Vorbis (CodecJOrbis) y gestiona la música ambiental y de discos.

### CodecMus

`class extends CodecJOrbis • 26 líneas`

**Decodificador de pistas de audio en formato .mus cifrado de C418.**

Aplica una clave de descifrado XOR para convertir el flujo de datos a audio Ogg reproducible.

> **Modding:** Necesario para decodificar las pistas de música oficiales del juego.

Métodos Clave:

- `InputStream openInputStream()`

### MusInputStream

`class extends InputStream • 55 líneas`

**Flujo de lectura con descifrado al vuelo para archivos de música .mus.**

Aplica una máscara de clave pseudoaleatoria para descifrar el audio OGG de C418 en tiempo de ejecución.

> **Modding:** Permite comprender el formato de empaquetado de música clásico de Mojang.

Métodos Clave:

- `int read()`
- `int read(byte abyte0[], int i, int j)`

### SoundManager

`class • 244 líneas`

**Gestor del sistema de audio y enlace con Paulscode SoundSystem (OpenAL).**

Inicializa OpenAL, reproduce música ambiental y emite efectos de sonido 3D con atenuación y paneo según la posición de la cámara.

> **Modding:** Llama a playSoundFX(...) para emitir cualquier efecto de audio personalizado.

Métodos Clave:

- `void func_340_a(GameSettings gamesettings)`
- `void onSoundOptionsChanged()`
- `void closeMinecraft()`
- `void func_6372_a(String s, File file)`
- `void addStreaming(String s, File file)`

## SoundPool

`class • 79 líneas`

**Colección de clips de sonido indexados por categoría y nombre.**

Mantiene listas de archivos de audio para cada acción (pasos, explosiones, golpes) y selecciona variaciones aleatorias de tono y volumen.

> **Modding:** Permite añadir nuevos efectos de audio al repertorio del juego.

Métodos Clave:

- `SoundPoolEntry addSound(String s, File file)`
- `SoundPoolEntry getRandomSoundFromSoundPool(String s)`
- `SoundPoolEntry getRandomSound()`

## SoundPoolEntry

`class • 19 líneas`

**Entrada individual de un archivo de sonido en un SoundPool.**

Almacena el nombre del sonido y la URL del archivo de audio en disco.

> **Modding:** Estructura interna del catálogo sonoro.

# 13. Matemáticas, Física y Estructuras de Datos (Math & Fast Collections) (11 clases)

Utilidades matemáticas y físicas de alto rendimiento. Cajas delimitadoras alineadas con los ejes (AxisAlignedBB) con piscina de reciclaje de memoria, vectores 3D (Vec3D), tablas trigonométricas precalculadas (MathHelper), algoritmos de ruido Perlin y tablas hash primitivas (MCHashTable).

## AxisAlignedBB

`class • 383 líneas`

**Caja delimitadora rectangular alineada con los ejes (AABB) con piscina de reciclaje de memoria.**

Modela volúmenes 3D (minX, minY, minZ a maxX, maxY, maxZ) para colisiones y selecciones. Incluye un pool estático de instancias para evitar saturar el Garbage Collector de Java durante miles de operaciones físicas por segundo.

**Modding:** Usa siempre AxisAlignedBB.getBoundingBoxFromPool(...) en lugar de 'new' para preservar el rendimiento.

Métodos Clave:

- `AxisAlignedBB getBoundingBox(double d, double d1, double d2, double d3, double d4, double d5)`
- `void clearBoundingBoxPool()`
- `AxisAlignedBB getBoundingBoxFromPool(double d, double d1, double d2, double d3, double d4, double d5)`
- `AxisAlignedBB setBounds(double d, double d1, double d2, double d3, double d4, double d5)`
- `AxisAlignedBB addCoord(double d, double d1, double d2)`

## HashEntry

`class • 63 líneas`

**Nodo interno para la tabla hash optimizada MCHashTable.**

Almacena la clave entera primitiva, el valor de referencia y el puntero al siguiente nodo colisionado.

**Modding:** Estructura interna de almacenamiento de alto rendimiento.

Métodos Clave:

- `int getHash()`
- `Object getValue()`
- `boolean equals(Object obj)`
- `int hashCode()`
- `String toString()`

## MCHashTable

`class • 166 líneas`

**Tabla hash personalizada optimizada para claves de tipo entero primitivo (int).**

Evita la sobrecarga de boxing de Integer a int de las colecciones estándar de Java. Crucial en gestión de entidades y chunks.

**Modding:** Usar en mods para mapear IDs numéricos a objetos con mínimo coste de memoria y CPU.

Métodos Clave:

- `Object lookup(int i)`
- `void addKey(int i, Object obj)`
- `Object removeObject(int i)`
- `void clearMap()`

## MathHelper

class • 86 líneas

**Tablas trigonométricas precalculadas y optimizaciones matemáticas.**

Mantiene tablas estáticas de seno y coseno (65536 entradas) que sustituyen las costosas llamadas a java.lang.Math. Incluye floor_double y sqrt rápidos.

> **Modding:** Usa MathHelper.sin() y MathHelper.cos() en cualquier cálculo de animación o física frecuente.

Métodos Clave:

- `float sin(float f)`
- `float cos(float f)`
- `float sqrt_float(float f)`
- `float sqrt_double(double d)`
- `int floor_float(float f)`

## MovingObjectPosition

class • 34 líneas

**Contenedor del resultado de una intersección por raycast (trazado de rayos).**

Almacena si la línea de visión o puntero impactó contra un bloque (coordenadas x, y, z y cara) o contra una entidad (entityHit).

> **Modding:** Consultar este objeto para saber qué bloque o criatura tiene seleccionada el jugador.

## NoiseGenerator

class • 13 líneas

**Clase base abstracta para generadores de ruido procedural.**

Base matemática para algoritmos de síntesis topográfica de terreno.

> **Modding:** Extender para implementar nuevos algoritmos de ruido matemático.

## NoiseGenerator2

class • 157 líneas

**Variante de generador de ruido procedural.**

Proporciona cálculos de modulación de terreno.

> **Modding:** Utilizado en la generación de biomas.

Métodos Clave:

- `void func_4157_a(double ad[], double d, double d1, int i, int j, double d2, double d3, double d4)`

## NoiseGeneratorOctaves

`class extends NoiseGenerator • 68 líneas`

**Generador de ruido fractal mediante suma de octavas de ruido Perlin.**

Combina múltiples capas de diferente frecuencia y amplitud para producir orografía natural y montañas creíbles.

> **Modding:** El componente más influyente en la forma de las cordilleras y valles de Minecraft.

Métodos Clave:

- `double func_806_a(double d, double d1)`
- `double[] func_807_a(double ad[], double d, double d1, double d2, int i, int j, int k, double d3, double d4, double d5)`
- `double[] func_4109_a(double ad[], int i, int j, int k, int l, double d, double d1, double d2)`

## NoiseGeneratorOctaves2

`class extends NoiseGenerator • 58 líneas`

**Generador fractal de octavas bidimensional.**

Optimizado para cálculos de mapas de calor y humedad en el plano horizontal X-Z.

> **Modding:** Determina la distribución geográfica de los biomas.

Métodos Clave:

- `double[] func_4112_a(double ad[], double d, double d1, int i, int j, double d2, double d3, double d4)`
- `double[] func_4111_a(double ad[], double d, double d1, int i, int j, double d2, double d3, double d4, double d5)`

## NoiseGeneratorPerlin

`class extends NoiseGenerator • 228 líneas`

**Implementación pura del algoritmo de Ruido Perlin tridimensional.**

Genera gradientes suaves de ruido a partir de una semilla numérica (seed).

> **Modding:** El ladrillo fundamental del generador de mundos de Notch.

Métodos Clave:

- `double generateNoise(double d, double d1, double d2)`
- `double lerp(double d, double d1, double d2)`
- `double func_4110_a(int i, double d, double d1)`
- `double grad(int i, double d, double d1, double d2)`
- `double func_801_a(double d, double d1)`

## Vec3D

`class • 206 líneas`

**Vector espacial tridimensional para cálculos cinemáticos y de posición.**

Modela posiciones y velocidades continuas (xCoord, yCoord, zCoord). Implementa suma, normalización, distancia euclidiana y producto escalar.

> **Modding:** Utilizado masivamente en físicas, raycasting de selección y trayectorias balísticas.

Métodos Clave:

- `Vec3D createVectorHelper(double d, double d1, double d2)`
- `void initialize()`
- `Vec3D createVector(double d, double d1, double d2)`
- `Vec3D subtract(Vec3D vec3d)`
- `Vec3D normalize()`